

Pruning Strategies of Vision Transformers: A Specialized Survey

SETAREH AHSAEI AND MOHSEN RAJI 

(Survey Paper)

School of Electrical and Computer Engineering, Shiraz University, Shiraz 84334-71949, Iran

CORRESPONDING AUTHOR: MOHSEN RAJI (email: mraji@shirazu.ac.ir).

ABSTRACT Vision Transformer (ViT) models are rapidly emerging as a preferred choice for solving various computer vision challenges. ViTs have a high parameter count, leading to substantial computational and memory demands, often exceeding those of traditional convolutional neural networks (CNNs). While compression techniques have proven effective in reducing these burdens, fundamental differences between CNNs and ViTs, such as architectural structure, feature extraction mechanisms, and input dependency, mean that CNN-based pruning schemes cannot be directly applied to ViTs. In this paper, a comprehensive examination of ViT pruning techniques is presented, making this the first survey focused exclusively on pruning in ViTs. Existing methods are analyzed from multiple perspectives, including pruning targets, granularity, pruning schedules, acceleration objectives, and selection criteria, to provide a structured understanding of their effectiveness. The differences between CNN and ViT pruning methods are elucidated, highlighting why traditional CNN strategies are insufficient for ViTs. A quantitative comparison of representative approaches is conducted to evaluate trade-offs among computational cost, model size reduction, and accuracy preservation. Finally, practical pruning guidelines are offered for deployment on resource-constrained devices, and future research directions are discussed to encourage further advancements in efficient, architecture-aware pruning strategies.

INDEX TERMS Pruning, Vision Transformer (ViT), model compression, model optimization, efficient AI models.

I. INTRODUCTION

Transformers have significantly advanced Natural Language Processing (NLP) by incorporating self-attention mechanisms that excel at capturing long-range dependencies in text [1]. This innovation has enabled them to surpass traditional models, such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, which struggled to process longer sequences efficiently [2]. The success of transformers in NLP has sparked interest in applying these models to vision-related tasks. Unlike CNNs, which rely on localized convolutions, ViTs use global self-attention, allowing them to model relationships across an entire image without inductive biases [3]. This makes ViTs more flexible and scalable, particularly for large datasets where capturing global context is critical. However, their performance, similar to that in NLP, improves with an increasing number of parameters.

For instance, the ViT-B/16 model, which contains 86 million parameters, achieves a top-1 accuracy of approximately 77.9% on ImageNet. Larger models, such as ViT-L/16, which has 307 million parameters, deliver even better performance, achieving a top-1 accuracy of around 87.76%. However, this enhanced performance comes at the cost of significantly higher computational demands [4]. This indicates that ViTs excel in tasks that require global feature learning; however, their efficiency is highly dependent on the model size.

ViTs, due to their large number of parameters, demand substantial computational resources and memory to efficiently process inputs [5]. This requirement makes them more resource-intensive compared to traditional models [6]. The computational demands of ViTs are notably high, making their deployment on resource-constrained devices challenging [7]. For example, applications like autonomous

TABLE 1. Representative Surveys and Descriptions

	Survey	Main Focus	Year
Vision Transformer	[1]	Compact architecture + Knowledge distillation + Quantization + Pruning	2022
	[2]	Knowledge distillation	2023
	[3]	Approximation + Knowledge distillation + Quantization + Pruning	2024
	[4]	Quantization	2024
	[5]	Compact architecture + Knowledge distillation + Quantization + Pruning	2024
Transformer	[6]	Tensor decomposition + Quantization + Pruning + NAS	2022
	[7]	Knowledge distillation + Quantization + Pruning + NAS	2023
	[8]	Compact architecture + Knowledge distillation + Quantization + Pruning	2024
	[9]	Quantization + Pruning	2024

drones [8], which require real-time object detection and rely on quick image processing, struggle to support these resource-hungry models. Edge devices, typically constrained by limited memory, lack the capacity to accommodate such large models, making direct deployment of uncompressed ViTs unfeasible [9]. This has resulted in an increasing interest in compressing ViT for deployment on devices with limited hardware resources, such as smartphones and embedded systems, where power and memory are constrained [10].

Various compression techniques have been proposed by researchers to develop more lightweight ViT models that can operate effectively on resource-constrained devices without significantly compromising performance [6]. These techniques include quantization, which reduces the precision of model weights and activations to lower bit widths [11]. Approximation methods, such as Linformer [12], simplify the self-attention mechanism to enhance computational efficiency. Furthermore, compact architectures, like Swin Transformer [13], are designed to reduce the computational cost of ViTs by minimizing or linearizing the complexity of the self-attention module, while still achieving strong performance [6]. Another popular method is knowledge distillation, where a smaller student model learns to imitate a larger teacher model, as in DeiT [14], achieving comparable performance with fewer resources [15]. Pruning, the most common compression approach, is a technique to remove unimportant parameters from neural networks to enhance efficiency without significantly compromising its performance. While pruning has seen less exploration in ViTs compared to other compression techniques, this research examines its potential to eliminate redundant weights, attention heads, tokens, or other elements, whether during training, post-training, or inference. By reducing the complexity of the model, pruning can lower memory usage and computational costs, often without compromising performance [16].

Numerous studies [17], [18], [19] have extensively examined pruning in DNNs from various perspectives. However, these works do not specifically address ViTs. Due to fundamental differences between CNNs and ViTs such as architectural structure, feature extraction mechanisms, and input dependency the categorization schemes used for CNN pruning

do not directly apply to ViTs. Table 1 presents a comprehensive overview of the primary work related to transformer and vision compression from previous surveys. Several studies, including [20], [21], [22], and [23] have reviewed earlier research on transformer compression. These studies generally address pruning in conjunction with other compression techniques; however, they do not concentrate on pruning as an independent strategy or analyze it in depth. Moreover, these reviews are not specifically centered on ViTs; instead, they provide a general overview of Transformer models, which limits their applicability to ViTs. In contrast, there have been several works specifically addressing ViT compression. For example, some studies focus on targeted techniques like knowledge distillation and quantization for ViT compression, as discussed in [24] and [25]. Additionally, other papers explore ViT pruning, offering insights into its potential for model reduction. However, many of these reviews, such as [26], [27] and [6] cover a broad spectrum of compression methods including pruning, quantization, and knowledge distillation without deeply analyzing each technique. For instance, [6] provides an overview of various compression approaches, including pruning, quantization, compact architecture, and knowledge distillation, but fails to address critical aspects of ViT pruning, such as pruning level, time to prune, criteria, or other essential pruning techniques that are crucial for efficient compression. Consequently, while these surveys offer useful guidance, they often overlook important details that could inform more specialized research in ViT-specific pruning strategies.

This paper has provided an in-depth analysis of pruning methods for ViTs, concentrating solely on pruning rather than broader compression techniques. Focusing exclusively on pruning has enabled an examination of its unique challenges and opportunities within the ViT architecture in greater detail. Briefly, the main contributions of this paper are as follows:

- *Comprehensive Examination of ViT Pruning Techniques:* To the best of our knowledge, this is the first survey dedicated solely to an in-depth analysis of pruning techniques applied to ViTs. It provides a detailed overview of ViT pruning methods, examines key approaches, and offers a focused analysis of this specific area of model compression.
- *Understanding the difference in CNN and ViT Pruning Methods:* This paper elucidates the fundamental differences between pruning techniques for CNNs and ViTs, highlighting why existing CNN pruning strategies cannot be directly applied to ViTs.
- *Evaluation of ViT Pruning from Multiple Perspectives:* Pruning techniques have been systematically assessed based on various factors, including the granularity of pruning, acceleration objectives, pruning stages, and selection criteria. This structured evaluation provides a deeper understanding of different pruning strategies and their effectiveness.
- *Quantitative Analysis and Efficiency Metric for Fair Comparison:* This paper presents a comprehensive quantitative analysis of various pruning methods in terms of

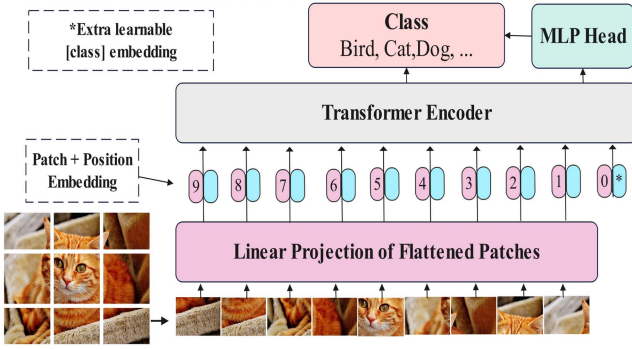


FIGURE 1. ViT architecture.

floating-point operations (FLOPs), parameters, and Top-1 accuracy.

- *Practical Pruning Guidelines for Model Designers and Potential Research Directions:* This paper offers insights into selecting appropriate pruning methods based on specific application requirements and computational constraints. Additionally, potential research directions have been outlined to encourage further advancements in ViT pruning.

The rest of the paper is organized as follows: Section II provides background on the ViT architecture. Section III discusses the motivation behind this work. Section IV introduces a framework for understanding pruning in ViTs. Section V explores the various components of ViTs that can be pruned. Section VI examines pruning granularity. Section VII focuses on pruning schedules. Section VIII outlines the criteria used for pruning decisions. Section IX presents a quantitative comparison of ViT pruning methods. Section X provides a discussion and outlines future directions. Finally, Section XI concludes the paper.

II. BACKGROUND

The ViT architecture consists of patch embedding and positional encoding to represent input images as sequences, a transformer encoder to process these sequences using self-attention mechanisms. Fig. 1 represents the ViT architecture.

A. PATCH EMBEDDING

Patch embedding in ViTs is a critical step for transforming images into a format compatible with the Transformer architecture. This process divides an input image $x \in \mathbb{R}^{H \times W \times C}$ where H , W and C denote the height, width, and number of channels, into $N = \frac{H \times W}{P^2}$ non-overlapping patches of size $P \times P$. Each patch is flattened into a vector of size $P^2 \cdot C$, then linearly projected into a D dimensional space using a learnable weight matrix $E \in \mathbb{R}^{(P^2 \cdot C) \times D}$:

$$z_0 = X_{Patch}E, \quad z_0 \in \mathbb{R}^{N \times D} \quad (1)$$

where X_{Patch} represents the flattened patches and z_0 is the resulting patch embedding sequence. This transformation enables ViTs to process visual data as sequential tokens, making

them compatible with the self-attention mechanism of the Transformer architecture. Moreover, patch embedding ensures that spatial and visual features are effectively preserved, facilitating the extraction of both local and global patterns.

B. POSITIONAL ENCODING

Positional encoding in ViTs is crucial for integrating spatial information into the sequence of image patches, compensating for the inherent lack of spatial awareness in the Transformer architecture [28]. Equation (2) represents this process:

$$z'_0 = z_0 + PE \quad (2)$$

where PE encodes the relative or absolute positions of the patches in the image, and z'_0 is the updated patch embedding that incorporates both the content and positional information. The addition of positional encoding allows the self-attention mechanism to capture spatial relationships across the image, enabling the model to better understand the underlying structure and context.

C. TRANSFORMER ENCODER

The Transformer encoder in ViT is composed of multiple stacked layers, each integrating a Multi-Head Self-Attention (MHSA) module and a position-wise Feedforward Network (FFN), along with residual connections and layer normalization to ensure stable training and efficient gradient propagation. MHSA is the core mechanism that enables the model to capture both local and global dependencies by allowing each image patch to attend to all other patches simultaneously.

Given an input token sequence $X \in \mathbb{R}^{N \times D}$, where N denotes the number of tokens (including image patches and the class token) and D represents the embedding dimension, three distinct representations are first generated through learned linear projections. Specifically, the query matrix $Q = XW_Q$ encodes the features used to query information from other tokens, the key matrix $K = XW_K$ represents reference features against which queries are compared, and the value matrix $V = XW_V$ contains the information to be aggregated and propagated. Here, W_Q , W_K , and $W_V \in \mathbb{R}^{D \times D}$ are learnable projection matrices that map the input embeddings into their respective subspaces.

Based on these representations, the self-attention operation computes pairwise interactions between tokens as:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

where the term QK^T measures the similarity between queries and keys, and d_k denotes the dimensionality of the query and key vectors. Scaling by $\sqrt{d_k}$ prevents excessively large dot-product values that could destabilize optimization. The softmax function normalizes the attention scores across all tokens, ensuring that the contributions of different patches are weighted proportionally. The resulting attention weights are then applied to the value matrix V , enabling the model to selectively aggregate information from relevant patches.

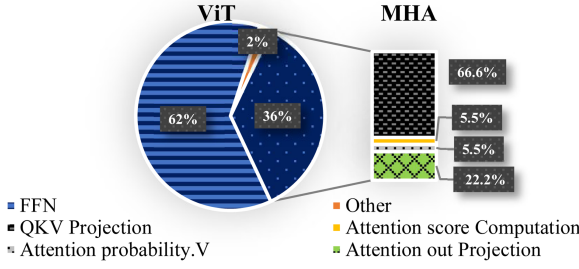


FIGURE 2. FLOPS analysis in ViT.

To enhance representational capacity, MHSA extends this operation by employing multiple attention heads. Each head independently learns attention patterns in a different subspace, allowing the model to capture diverse semantic relationships. The outputs of all h heads are concatenated and linearly projected as:

$$MHSA(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O \quad (4)$$

where $\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$ corresponds to the output of the i -th attention head, and $W_O \in \mathbb{R}^{hD \times D}$ is a learnable output projection matrix that maps the concatenated features back to the original embedding dimension. This multi-head formulation enables the Transformer encoder to jointly attend to information from different representation subspaces, which is essential for modeling complex visual structures in ViTs.

Following MHSA, a position-wise FFN is applied to each token independently, expressed as:

$$FFN(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 \quad (5)$$

where x is the input token (output of the MHSA with residual connection), $W_1 \in \mathbb{R}^{D \times D_{ff}}$ and $b_1 \in \mathbb{R}^{D_{ff}}$ are the weight matrix and bias of the first linear layer, $W_2 \in \mathbb{R}^{D_{ff} \times D}$ and $b_2 \in \mathbb{R}^D$ are the weight matrix and bias of the second linear layer, and GELU is the Gaussian Error Linear Unit activation function. This FFN enables complex, token-wise transformations, enhancing the expressiveness of the model.

III. MOTIVATION

In ViTs, the majority of the computational burden is measured in terms of FLOPs. Fig. 2 represents the ViT-B/16 model. The FFN accounts for approximately 62% of the total FLOPs, followed by the MHA mechanism at 36%, with the remaining 2% attributed to other operations. Within the MHA, the most computationally expensive components are the query, key, and value (QKV) projections, consuming 66.6% of the MHA's FLOPs. The attention output projection accounts for 22.2%, while attention score computation and attention probability vector calculation contribute 5.5% and 5.5%, respectively [29]. This analysis highlights that, similar to CNNs and DNNs, the computational cost of ViTs is largely driven by matrix multiplication operations.

At first glance, this computational similarity suggests that pruning techniques developed for CNNs may be applicable to ViTs. However, despite sharing comparable sources of computational cost, ViTs differ fundamentally from CNNs in their architectural design and information processing mechanisms,

which significantly impacts how pruning should be performed. One key difference lies in input dependency. CNNs process images using fixed convolutional kernels that generate feature maps in a consistent manner for all inputs, enabling pruning strategies such as filter or channel pruning to rely on uniform activation behavior [30]. In contrast, ViTs exhibit dynamic input dependency, where the importance of each token varies across different images [31]. The self-attention mechanism computes token-to-token interactions based on the specific content of each input, making the relative importance of tokens and attention heads highly input-dependent [32]. This dynamic behavior complicates pruning, as the relevance of components cannot be assumed to be fixed across samples.

A second fundamental distinction is the difference between local and global feature extraction. CNNs are inherently designed to capture local spatial patterns through localized receptive fields, enabling effective extraction of edges, textures, and other low-level features. ViTs, on the other hand, rely on global self-attention, allowing each token to attend to all other tokens across the image and model long-range dependencies [33]. While this global context modeling is a key strength of ViTs, it also introduces new challenges for pruning, as removing tokens or attention heads may disrupt critical long-range relationships that are less pronounced in CNNs due to their localized processing.

Finally, ViTs exhibit a heterogeneous architecture, in contrast to the relatively homogeneous structure of CNNs [34]. CNNs are typically composed of repeating convolutional layers with similar structural properties, which facilitates the application of uniform pruning strategies across the network. ViTs, however, consist of distinct components, including patch embedding layers, MHSA modules, and FFNs, each playing a unique role in representation learning. Pruning these components affects the model in different ways; for example, pruning attention heads may impair global dependency modeling, whereas pruning FFN layers primarily impacts dense feature transformations. This architectural heterogeneity necessitates component-specific pruning strategies rather than a single uniform approach.

Although ViTs and CNNs share similarities in terms of computational bottlenecks, the dynamic input dependency, global attention mechanism, and heterogeneous architecture of ViTs render CNN-oriented pruning techniques and taxonomies insufficient. Existing pruning surveys predominantly focus on CNNs or treat pruning as a secondary component of broader transformer compression strategies, failing to systematically analyze ViT-specific pruning challenges. These limitations motivate the need for a dedicated and structured survey that focuses exclusively on pruning in Vision Transformers. By explicitly addressing the unique characteristics of ViTs, such a survey is essential for providing clear insights, practical guidance, and a unified understanding of pruning strategies tailored to ViT architectures.

IV. A FRAMEWORK FOR UNDERSTANDING VIT PRUNING

Pruning techniques in ViTs can be analyzed from multiple perspectives, each addressing a critical design choice for

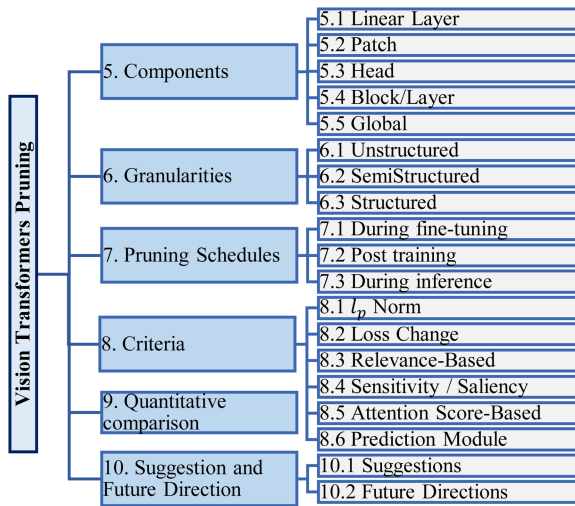


FIGURE 3. Overview of ViT Pruning Strategies (the numbers represent the section addressing each topic).

model optimization. The following sections explore these perspectives, considering key decisions faced by model designers. The Overview of ViT pruning strategies is illustrated in Fig. 3.

What components of ViTs can be pruned? Unlike CNNs, ViTs introduce unique structural elements that allow for multiple pruning strategies. The primary components subject to pruning include: Linear layer pruning, Patch pruning, Attention head pruning, Transformer block pruning and global pruning that considers all types of pruning.

These pruning strategies align with the modular nature of ViTs, offering flexibility in optimizing models for different computational constraints and accuracy requirements.

What are the different granularities of pruning in ViTs? Pruning techniques in ViT can be categorized based on the granularity at which parameters are removed. The level of granularity impacts both the computational efficiency and the compatibility of the pruned model with various hardware architectures. The three primary categories are:

- **Unstructured Pruning:** Removes individual weights without structural constraints, offering high sparsity but requiring specialized hardware for acceleration.
- **Semi-Structured Pruning:** Applies sparsity in a more regularized way, such as block or pattern-based pruning, balancing flexibility and computational efficiency.
- **Structured Pruning:** Eliminates entire components like attention heads, neurons, or layers, leading to direct speedups and better hardware compatibility.

The choice of pruning granularity depends on the trade-off between model efficiency, hardware compatibility, and performance retention. While unstructured pruning provides finer control over sparsity, structured pruning ensures better real-world acceleration, and semi-structured pruning offers a middle ground by balancing flexibility and efficiency.

When should pruning be applied? Pruning in ViTs can occur at various model development stages, including during training, after training, or during inference. Pruning during training allows for dynamic adjustments based on the model's evolving performance, potentially leading to more effective reductions in unnecessary complexity. Alternatively, pruning post-training can be advantageous as it utilizes a fully trained model to inform which parameters or tokens can be removed without significantly affecting accuracy. Pruning during inference involves real-time decisions based on the current input and resource constraints, enabling adaptive optimization of the computational load of the model. The chosen stage should align with the specific goals and deployment constraints of the application.

What is the most effective pruning criterion for a given task? The selection of the most effective pruning criterion for a specific task in ViTs depends on several factors, including the nature of the task, architecture of the model, and the performance metrics that are prioritized. ViT pruning criteria include:

- The l_p Norm, which prunes elements with lower norms,
- Sensitivity-based methods, which assess the impact of removing certain parameters or tokens on model performance,
- The loss change criterion for pruning evaluates the impact of removing specific elements by measuring the change in loss,
- Relevance-based methods, which identify and retain the most important features for the task at hand,
- Attention score-based criteria, which utilize the attention weights to determine the significance of elements.
- Predictive modules, evaluate the impact of element pruning on final performance by utilizing a predictive model, such as a Multi-Layer Perceptron (MLP) or CNN.

Ultimately, the optimal criterion will vary based on the specific requirements of the task, necessitating an evaluation of multiple criteria to identify the most suitable approach for maximizing model efficiency and effectiveness.

At the end, various pruning methods are discussed, and promising directions for future research are proposed.

V. PRUNING COMPONENTS IN VITS

Neural pruning refers to the process of compressing a neural network by removing parameters to find a smaller “subnetwork” that retains the original performance [35]. Pruning in ViTs involves reducing the number of parameters or computations by removing less important components of the model. ViTs are based on a transformer architecture that uses self-attention mechanisms [36]. This introduces various unique components like MHSA, token embeddings, and dense FFNs, each of which can be pruned at different levels of granularity.

Pruning in ViTs can be applied at various levels of the architecture, from fine-grained techniques like pruning individual weights to coarser-grained methods such as removing entire transformer blocks or layers. These pruning approaches are

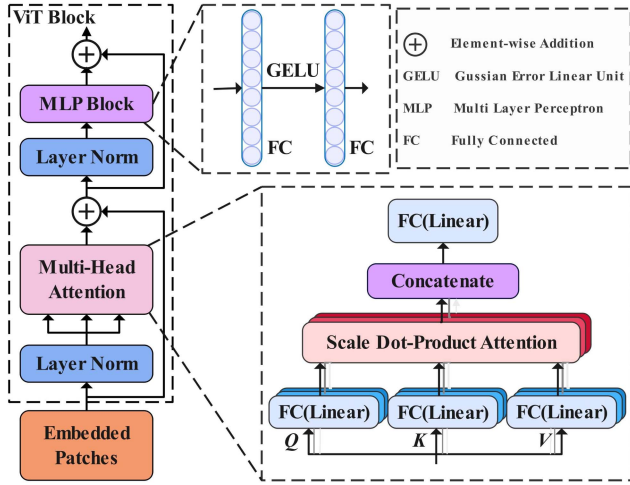


FIGURE 4. FCs in transformer block.

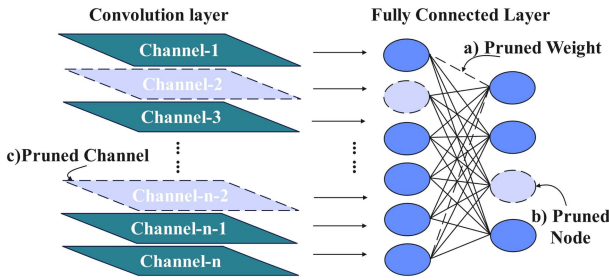


FIGURE 5. Different level of linear layer pruning in ViT.

categorized based on the specific part of the network being pruned, including: linear layer pruning, patch pruning, head pruning, and block pruning.

A. LINEAR LAYER PRUNING

Fig. 4 illustrates that each ViT block contains a total of six learnable linear projections and two LayerNorm modules. The linear layers in ViTs are responsible for processing the output of the self-attention mechanism and token embeddings and thus, they are often the most time-consuming part of the model [37]. Fig. 5 illustrates the different levels of linear layer pruning in ViTs, including weight, neuron, and channel pruning.

1) WEIGHT PRUNING

This involves removing individual weights within the linear layers based on their magnitude or contribution to the final output [38]. Weight pruning creates sparsity in the model but typically requires specialized hardware to achieve computational savings during inference [39]. This method can apply to the whole network, not limited to linear layers.

A recent approach was introduced by Frantar et al [40], who introduced the Optimal ViT Surgeon (oViT), a weight pruning method designed specifically for ViTs. oViT performs unstructured weight pruning using second-order information,

allowing it to account for weight correlations and maintain high accuracy even at substantial sparsity levels.

The scoring of weights in oViT is based on the Optimal Brain Surgeon (OBS) framework [41], which utilizes second-order information to estimate how removing a weight affects the loss function. The saliency score for each weight is calculated as:

$$\rho_i = w_i^2 / 2 [H_L^{-1}(w^*)]_{ii} \quad (6)$$

where ρ_i is the saliency score for the i -th weight w_i , and $H_L^{-1}(w^*)$ is the inverse of the Hessian matrix at the optimal weights w^* . Equation (6) accounts for weight correlations and helps identify which weights can be pruned with minimal impact on overall performance of the model. Weights with the lowest saliency scores are pruned first, and the remaining weights are updated accordingly to compensate for the changes, ensuring that accuracy is maintained.

Prasetyo et al. [42] introduced the Sparse-ViT to enhance the efficiency of ViTs by integrating sparse regularization with unstructured pruning. This approach starts with applying sparse regularization during the training phase, which encourages many weights to become near-zero, thus making it easier to identify unimportant weights. Following training, unstructured pruning is applied to remove these less critical weights, thereby reducing the overall complexity of the model. Pruning is applied globally across the ViT, utilizing the L1-norm to select and eliminate the least important weights. This two-step approach, first applying sparse regularization, then using L1-norm-based pruning, strikes a balance between maintaining high computational efficiency and preserving accuracy. The results demonstrate that models with this sparse regularization technique outperform those without it, particularly in terms of both accuracy and efficiency when higher pruning ratios are used.

Ahsaei et al. [43] introduced LOST-ViT, a pruning-based framework that improves both efficiency and fault tolerance in ViT. Their approach applies saliency-driven weight pruning to remove less important parameters, reducing model size and computational cost while simultaneously limiting the number of weights exposed to hardware faults. This pruning step forms the basis for a lightweight soft-error mitigation strategy, enabling ViTs to be more robust and compact for deployment in resource-constrained and safety-critical systems.

Shi et al. [44] proposed Unified and Progressive Pruning (UPop), a framework for compressing vision-language transformers by adaptively assigning pruning ratios across all components, including self-attention, FFNs, and cross-attention layers. UPop combines a unified search for component importance with progressive pruning, gradually driving weights toward zero while retraining, which improves convergence and maintains performance at higher compression ratios. This approach outperforms mask-based and iterative methods, achieving efficient compression and high accuracy in multimodal tasks such as image-captioning, visual question answering, and image-text retrieval.

The proposed method by Yang He et al. [45], named as Data-Independent Module-Aware Pruning (DIMAP), a post-training weight pruning method designed for hierarchical ViTs such as Swin Transformer. Unlike magnitude-based pruning, DIMAP evaluates weight importance by measuring information distortion, enabling fair comparison across different layers and modules. The method independently prunes attention and MLP modules using a Frobenius distortion criterion, retaining weights that contribute most to model performance. Without requiring training data, DIMAP achieves substantial parameter and FLOPs reduction with minimal accuracy degradation, even under high pruning ratios.

2) NEURON PRUNING

Neural pruning involves removing entire neurons from the FFNs to reduce complexity and computational cost while preserving overall model performance [46].

Shim et al. [47] introduced Structured Neuron-level Pruning (SNP), a post-training pruning method for Vision Transformers that compresses the model while preserving attention behavior. SNP operates at the neuron level within the multi-head self-attention (MSA) module and focuses on removing redundant filters in the value layers without disrupting attention scores. The core idea is to eliminate inter-head redundancy by identifying filters that provide overlapping information across different attention heads. To quantify this redundancy, SNP computes an importance score for each filter based on its similarity to filters in other heads, defined as:

$$w_{v,i}^h = \sum_{j=1}^H \sum_{l=1}^{d_v} (1 - |\cos(W_i^h, W_l^j)|) \quad (7)$$

where $w_{v,i}^h$ denotes the importance score of the i -th filter in the value layer of the h -th attention head, H is the total number of heads, and d_v is the number of filters in each value layer. The term $\cos(W_i^h, W_l^j)$ measures the cosine similarity between the target filter and filters from all heads, with lower similarity indicating lower redundancy. Filters with smaller importance scores are considered highly redundant and are pruned accordingly. By removing these redundant neurons, SNP reduces model complexity while maintaining accuracy, and its hardware-agnostic design enables practical speedups on standard platforms without requiring specialized libraries.

3) CHANNEL PRUNING

Similar to CNNs, channel pruning can be applied to the linear layers in ViTs, particularly in the FFNs. Channels correspond to the different dimensions in the input/output of the linear layer, and pruning them reduces the size of the entire input or output vector [48].

Zhu et al. [49] introduced the ViT pruning (VTP) method, which uses channel pruning to reduce the computational cost and memory of ViTs. The method applies L1 regularization during training to promote sparsity in the importance scores

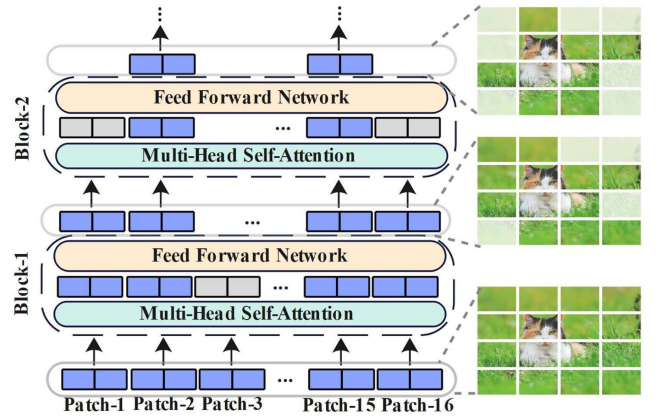


FIGURE 6. Patch pruning in ViT.

of channels, identifying which ones contribute most to performance. Channels with low importance scores are pruned, and the model is fine-tuned to recover any accuracy loss. This pruning is applied to both the MSA and MLP blocks, significantly reducing the number of parameters and FLOPs while maintaining high accuracy, making VTP an efficient solution for deploying ViTs on resource-constrained devices.

Ankur Kumar [50] proposes a hybrid compression method for ViTs that combines channel pruning and low-rank approximation to reduce model size while maintaining accuracy. The method focuses on channel pruning, where unimportant rows or columns from the feedforward layers are pruned, and low-rank approximation is applied to compress the query, key, and value matrices in the self-attention layers. During training, the importance of each dimension is determined by learning a mask that assigns lower values to less important dimensions, guided by L1 regularization to encourage sparsity. At inference time, dimensions with mask values below a set threshold are pruned, effectively reducing model parameters with minimal performance loss.

Lee and Hwang [51] introduced an automatic channel pruning method for MSA (APMA) mechanisms in transformers, aimed at reducing the computational complexity of ViTs. The method addresses the challenge of channel misalignment by incorporating similarity-based weights into the pruning indicator to ensure that informative channels are preserved. Additionally, it enforces equal channel pruning across all heads, preventing the removal of entire representation subspaces in the MHSA module. A reweight module is applied to compensate for any information loss caused by the pruning process, allowing remaining channels to retain the lost information.

B. PATCH PRUNING

In ViTs, the quadratic time and memory complexity are associated with the number of input patches [52]. Fig. 6 illustrates

the process of patch pruning in the ViT. Patch pruning (or token pruning) involves removing certain image patches before they enter the self-attention mechanism.

The method proposed by Liang et al. [53] is EViT introduces a patch pruning technique to accelerate ViTs by reorganizing and reducing the number of image patches processed. In ViTs, redundant computations arise as many patches, particularly those containing backgrounds or less relevant information, do not significantly contribute to predictions. EViT addresses this by identifying and preserving the most attentive patches based on their interaction with the CLS token while fusing less important patches into a single token, thereby reducing computation. This dynamic patch pruning occurs progressively during training, improving inference speed without requiring additional parameters or altering the ViT architecture. Through this effective patch pruning technique, EViT strikes a favorable balance between accuracy and computational efficiency, making it suitable for real-world applications.

DynamicViT, introduced by Rao et al. [54], aims to enhance the efficiency of ViTs by implementing dynamic token sparsification. Unlike methods that focus on pruning specific weights or neurons, DynamicViT focuses on progressively pruning tokens during inference, selecting which tokens to retain based on their importance to the final prediction. This is achieved using lightweight prediction modules that dynamically assess the importance of each token in real time. Based on these assessments, the method generates binary decision masks that identify and prune uninformative tokens. These masks are applied in a hierarchical manner across multiple layers, allowing for a gradual increase in the number of pruned tokens as the network processes the input. To ensure that pruned tokens do not influence the remaining ones, an attention masking strategy is employed, which blocks the interactions of pruned tokens within the attention matrix.

Dong et al. [55] introduced HeatViT, a hardware-efficient and image-adaptive token pruning framework aimed at reducing the computational and memory overhead of Vision Transformers while preserving accuracy. HeatViT dynamically identifies non-informative tokens before the transformer blocks using an attention-based multi-head token selector, enabling input-dependent pruning tailored to each image. Each attention head evaluates both local and global token features and assigns a token importance score, which are aggregated to generate a decision mask that determines whether tokens are retained or pruned. To mitigate information loss caused by aggressive pruning, HeatViT introduces a token packaging strategy that summarizes discarded tokens into a single representative token rather than removing them entirely. Given T non-informative tokens $\{\hat{x}_t\}_{t=1}^T \in \mathbb{R}^{T \times D}$ and their corresponding token scores $\{\tilde{s}_t\}_{t=1}^T \in \mathbb{R}^{T \times 2}$, the package token is computed as:

$$P = \frac{\sum_{t=1}^T \hat{x}_t * \tilde{s}_t [0]}{\sum_{t=1}^T \tilde{s}_t [0]} \in \mathbb{R}^{1 \times D} \quad (8)$$

where P denotes the package token, $\tilde{s}_t [0]$ represents the keep score of the t -th token predicted by the classifier, and $*$ indicates element-wise multiplication. This weighted averaging preserves critical information from non-informative tokens and allows the package token to participate in subsequent transformer layers alongside informative tokens. By combining adaptive token pruning with token packaging, HeatViT achieves substantial efficiency gains and is particularly well suited for deployment on resource-constrained edge devices such as FPGAs.

Yehui Tang et al. [56] proposed Patch Slimming, a post-training patch-level pruning method that improves ViT efficiency by removing redundant image patches. Instead of pruning weights or channels, Patch Slimming evaluates the impact of each patch on the final class token and discards less informative patches using a top-down strategy, starting from the final layer and propagating important patch selections backward across layers. By pruning patches with low contribution to classification, the method significantly reduces sequence length and computational cost. Experimental results show that Patch Slimming can eliminate over 45% of FLOPs with minimal accuracy degradation, making it well suited for efficient ViT inference on resource-constrained devices.

Quan Tang et al. [57] proposed Dynamic Token Pruning (DToP), a token-level pruning method for ViT-based semantic segmentation that reduces computation by dynamically removing “easy” tokens during inference. DToP adopts a coarse-to-fine strategy, where tokens with high confidence are finalized early and excluded from deeper transformer stages, while low-confidence (“hard”) tokens continue processing. Token difficulty is estimated using auxiliary blocks and a confidence threshold, with top- k tokens per semantic class preserved to maintain contextual integrity. This dynamic pruning scheme achieves up to 35% FLOPs reduction with negligible impact on segmentation accuracy.

The Token Pruning & Pooling Transformers (PPT) model, proposed by Xinjian Wu et al. [58], optimizes ViTs by addressing two types of redundancy: inattentive redundancy and duplicative redundancy. Inattentive redundancy involves tokens that contribute little to the final output, while duplicative redundancy arises from similar tokens in shallow layers. PPT adaptively applies token pruning at deeper layers where token importance becomes more distinct and token pooling at shallower layers where tokens exhibit higher similarity. This adaptive strategy ensures a balanced reduction of computational cost while maintaining model accuracy, with no need for additional trainable parameters. The combination of token pruning and pooling reduces FLOPs and improves throughput without sacrificing accuracy, making it a practical solution for efficient ViT deployment in real-world tasks.

The proposed method by Siyuan Wei et al. [59] introduces a novel Token Pruning & Squeezing (TPS) module aimed at improving the efficiency of ViTs by aggressively compressing their computational requirements. The TPS module performs two main tasks: pruning less significant tokens and squeezing the information from pruned tokens into the retained ones,

thus mitigating the information loss typically associated with token pruning. The solution targets the token weights within the self-attention layers of the ViT, pruning redundant tokens while preserving a subset of important ones. The pruning is applied based on the importance scores of tokens, which are computed using the cosine similarity between the token features. The scoring function identifies tokens to keep using the Top-k operation, and pruned tokens are matched and fused with the retained tokens using a unidirectional nearest-neighbor algorithm. The pruning score for each weight is calculated as:

$$c_{i,j} = x_i^T x_j / (\|x_i\| \|x_j\|) \quad (9)$$

where $c_{i,j}$ represents the similarity between a pruned token x_i and a reserved token x_j guiding the assignment of pruned tokens to the nearest reserved tokens for information assimilation. This method preserves the overall structure while ensuring a constant token shape, enabling hardware-friendly inference.

Yunke Wang et al. [60] propose the Multi-Tailed ViT (MT-ViT), which improves ViT efficiency by dynamically adjusting the number of input patches. MT-ViT generates multiple "tails," each producing visual sequences of different lengths, and a tail predictor selects the appropriate sequence based on image complexity. By optimizing the image-to-token step rather than pruning within Transformer layers, MT-ViT reduces computational cost upfront while sharing a common encoder across tails, maintaining accuracy with lower FLOPs during inference.

The Selective ViT (SViT) method, proposed by Yifei Liu et al. [61], revisits token pruning for object detection and instance segmentation tasks using ViTs. SViT preserves pruned tokens in the feature maps for later reuse. It introduces four key innovations: (i) pruned tokens are reused on-demand in later layers, (ii) a dynamic pruning rate adjusts the number of pruned tokens based on image complexity, (iii) a lightweight 2-layer MLP efficiently prunes tokens, and (iv) reactivating pruned tokens improves performance by adapting to layer-specific attention. SViT employs a simple 2-layer MLP followed by Gumbel Softmax to determine which tokens to prune, with the dynamic pruning rate controlled during training. This approach balances computational efficiency and performance, reducing loss and increasing inference speed, without significantly impacting accuracy on dense tasks like object detection and segmentation.

The Synergistic Patch Pruning (STAR) method, proposed by Yuyao Zhang et al. [62], a training-free patch pruning method that reduces ViT computation by progressively removing less important image patches. STAR evaluates patch importance using a dual strategy: intra-layer importance, computed online from CLS-token attention scores to capture layer-specific relevance, and inter-layer importance, computed offline via Layer-wise Relevance Propagation (LRP) to preserve patches critical to deeper layers. These two scores are fused by minimizing their Kullback–Leibler divergence, producing a unified pruning score that ranks patches for removal.

By synergistically combining local and global importance, STAR achieves significant FLOPs reduction and inference speedup while maintaining accuracy close to the original ViT models, without requiring retraining.

Xu et al. [63] propose the Graph-based Token Propagation (GTP) method. GTP identifies less important tokens based on attention-derived importance scores, reflecting both their contribution to information flow and their ease of regeneration. Instead of directly discarding these tokens, GTP propagates their information to more important tokens using a graph-based formulation governed by a normalized adjacency matrix. This structured token propagation effectively condenses the token sequence while retaining representational capacity. By pruning token-associated computations without modifying model parameters or requiring retraining, GTP achieves substantial computational savings with minimal impact on accuracy and interpretability.

Lee et al. [64] proposed a method, called DCT-ViT, utilizes the Discrete Cosine Transform (DCT) to prune high-frequency patches in ViTs to reduce computational costs while preserving model accuracy. It focuses on pruning patches in both spatial and channel dimensions using 1D-DCT, identifying and removing unnecessary high-frequency components that contribute less to image recognition. This pruning occurs progressively during training, with static removal of patches, eliminating the need for additional modules or dynamic decisions during inference. By transforming the patches into the frequency domain through DCT, high-frequency elements are pruned according to a predefined ratio, allowing the model to retain essential low-frequency information. This approach significantly reduces computational costs with minimal impact on accuracy, making it efficient for deployment in resource-constrained environments.

The Token Fusion (ToFu) method, proposed by Minchul Kim et al. [65], a hybrid token reduction method that unifies token pruning and token merging in ViTs. ToFu dynamically selects between pruning and merging based on the functional linearity of each transformer layer, which reflects the sensitivity of the output to input interpolation. Token pruning is applied in early layers with low linearity, while token merging is favored in later layers where representations are more stable. To preserve feature distributions during merging, ToFu introduces Maximum Norm Linear Interpolation (MLERP), and employs Bipartite Soft Matching (BSM) to identify similar tokens. By adaptively combining pruning and merging, ToFu achieves substantial token reduction while maintaining accuracy, outperforming prior methods on tasks such as ImageNet classification and image generation.

Zhenglun Kong et al. [66] proposed Soft Pruning ViT (SPViT) method, a dynamic patch pruning method that evaluates patch importance via attention scores from multiple heads. Less informative patches are merged into a package patch rather than discarded, preserving essential background information. SPViT incorporates a latency-aware sparsity loss during training to meet edge-device constraints, enabling

adaptive, per-image patch pruning for efficient real-time inference.

Patch pruning can be combined with other pruning techniques, with linear layer pruning being one of the most commonly used methods alongside it.

1) PATCH + NEURON

Hancheng Ye et al. [67] proposed Once for Both (OFB) method, a single-stage ViT compression method that jointly evaluates importance and sparsity scores to guide pruning. The method aims to streamline the compression process by entangling importance and sparsity evaluations, thereby reducing the complexity typically associated with separate evaluation stages. OFB operates during the model search stage and introduces a bi-mask weight-sharing strategy to assess unit prunability, calculating a combined score for each unit using both the importance score and a sparsity score that are optimized together. The search process is regularized using an adaptive one-hot loss function to progressively refine the pruning decisions, while a Progressive Masked Image Modeling (PMIM) technique enhances the feature space during compression. This approach results in more efficient and effective pruning compared to traditional two-stage methods.

2) PATCH + CHANNEL

Ao Wang et al. [68] proposed method compresses ViTs using two complementary strategies: Asymmetric Token Merging (ATME) and Consistent Dynamic Channel Pruning (CDCP). ATME reduces the number of tokens by merging adjacent tokens horizontally and vertically, which preserves the spatial structure of images and ensures the model remains effective for tasks like semantic segmentation. CDCP dynamically prunes unimportant channels in MSA modules during fine-tuning, enabling fine-grained optimization of weights of the model. This pruning happens after pretraining, ensuring that channels with minimal impact on performance are removed, improving both speed and efficiency without degrading accuracy. The importance of channels is scored during fine-tuning, and the least important channels are progressively pruned while maintaining consistency across different heads and attention layers.

3) PATCH + WEIGHT

Dhruv Parikh et al [69] proposed a combined approach of static weight pruning and dynamic token pruning to accelerate ViT inference on FPGA, effectively reducing both model size and computational complexity. Static weight pruning eliminates redundant weights from key components such as the queries, keys, values, and projection weight matrices within the MSA and MLP layers. Simultaneously, dynamic token pruning identifies and removes less important tokens based on attention scores during inference, reducing the computational load. This approach is implemented both during training, where the model is fine-tuned to recover from accuracy loss caused by pruning, and during inference, where the number

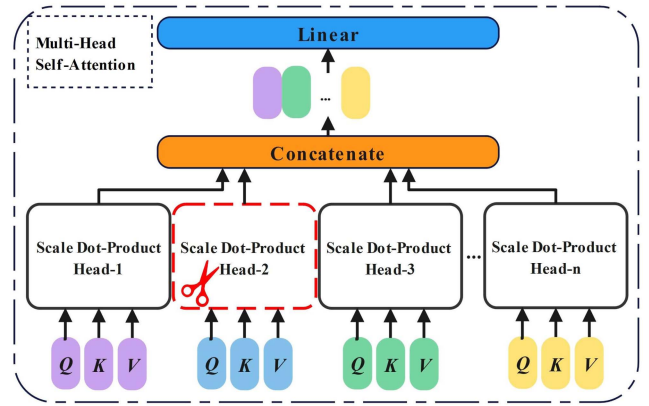


FIGURE 7. Head pruning in ViT.

of tokens is dynamically adjusted. The pruning decisions are based on a block-wise scoring matrix, where the importance of weight blocks is calculated, and unimportant blocks are masked out. This integrated algorithm-hardware co-design enables efficient execution of pruned ViT models on FPGA, achieving significant reductions in computation complexity while maintaining competitive model performance.

C. HEAD PRUNING

Fig. 7 illustrates the head pruning process within the ViT. The multi-head self-attention mechanism is a core component of ViTs, where multiple heads are used to capture different relationships between tokens [70]. Head pruning involves removing entire attention heads that contribute the least to output of the model. Since each head operates independently, this form of structured pruning reduces the number of computations in the attention mechanism without significantly affecting capacity of the model to capture relationships between tokens [37].

Head pruning can be paired with other pruning techniques, with linear layer pruning being one of the most commonly employed methods alongside it.

1) HEAD + NEURON

Chuangyang Zheng et al. [34] introduced Structure-Aware ViT Pruning (SAViT), a framework designed to prune both MHSA heads and FFN neurons to reduce computational costs. This method integrates head pruning and neuron pruning while also considering the embedding dimensions. What distinguishes SAViT is its collaborative optimization approach, which accounts for the interactions between these components, thereby ensuring a more balanced pruning process. The pruning occurs during training, utilizing a Taylor-based approximation to evaluate both the individual importance of each component and their interactions. This joint analysis guarantees that less important weights, attention heads, and neurons are pruned, significantly decreasing the computational load (such as FLOPs) while maintaining high performance across various tasks, thus ensuring minimal impact on model accuracy.

Lu Yu et al. [71] proposed X-Pruner method, an explainable pruning framework for ViTs that optimizes efficiency while preserving accuracy. The method introduces an explainability-aware mask that quantifies the contribution of each prunable unit, such as attention heads or neurons in MSA and FFN layers, to class predictions. The mask is learned end-to-end during training, enabling adaptive, differentiable pruning that automatically removes units with low importance while retaining critical components. By varying the pruning rate across layers and focusing on class-relevant units, X-Pruner achieves a more compact model with reduced computational costs, making it suitable for deployment on resource-constrained devices without compromising performance.

Miao Yin et al. [72] proposed GOHSP, a unified framework that uses graph-based and optimization-based structured pruning to reduce the size and computational cost of ViTs. The method focuses on pruning MHSA heads and columns of MLPs to introduce structured sparsity in a hardware-friendly way. GOHSP prunes both heads and neurons, addressing the inherent architectural heterogeneity in ViTs by applying different sparsity patterns to the attention and MLP layers.

GOHSP operates during training, first using a graph-based ranking method to measure the importance of attention heads by modeling inter-head correlations as a Markov chain. The importance of the attention heads is determined by calculating the transition probability between two attention heads as:

$$P(i, j) = \text{CosineSim} \left(\sum_{b=1}^B A_i(X_b), \sum_{b=1}^B A_j(X_b) \right) \quad (10)$$

where $A_i(X_b)$ and $A_j(X_b)$ represent the output of i -th and j -th attention heads on a batch of input data X_b . This probability score ranks the heads, guiding the pruning process.

After ranking, GOHSP uses a soft pruning strategy where less important heads and columns are flagged for removal, and the final pruning is performed through an optimization-based process. This approach imposes sparsity gradually, allowing the model to maintain performance while reducing parameters and computational load efficiently.

Hao Yu and Jianxin Wu [37] introduced Unified Pruning ViT (UP-ViT), a framework designed to reduce the computational cost of ViTs by pruning multiple key components, including MSA heads, FFN neurons, and channels within residual connections and normalization layers. The framework maintains overall structure of the model, ensuring consistency in token representation, which allows it to generalize effectively across various tasks such as object detection and classification. The pruning process occurs during training, where the importance of each channel, head, or neuron is evaluated by comparing the outputs of the pruned and original models. This comparison identifies less important components, which are then pruned. After pruning, the model is fine-tuned, and knowledge distillation is applied to help the pruned model retain accuracy. This method significantly reduces both size of the model and computational cost, ensuring

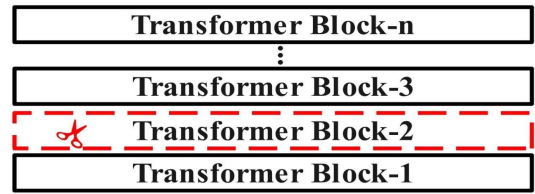


FIGURE 8. Block pruning in ViT.

high accuracy while making ViTs more efficient for practical use in resource-constrained environments.

2) HEAD + PATCH

The method proposed by Zhuoran Song et al. [29], called CP-ViT, is a pruning technique for ViTs that incrementally reduces computational complexity by identifying and removing redundant patches and heads. This approach specifically targets MHA and FFN components, which are significant contributors to the computational cost in ViTs. CP-ViT prunes these components by dynamically predicting sparsity levels across the network with the objective of maintaining model accuracy while reducing computational demands. CP-ViT quantifies the informativeness of each patch and head by using attention values to calculate cumulative scores, thereby identifying regions that contribute less to the overall model output. The pruning method then progressively removes the least informative areas, adapting pruning ratios based on specific attention range of each layer to minimize any loss of accuracy. Additionally, pruning masks are applied to both MHA and FFN inputs, effectively skipping computations in pruned regions to enhance efficiency while preserving performance of the model.

D. BLOCK OR LAYER PRUNING

ViT consist of multiple layers (or blocks), each comprising MHSA followed by FFN [73]. Fig. 8 shows the block pruning process in the ViT. Block or layer pruning involves removing entire transformer layers to reduce the depth of the model.

Fang Yu et al. [74] introduces a Width & Depth Pruning (WDPruning) framework to reduce the computational cost of ViTs by pruning both the width and depth. Width pruning focuses on removing less important heads in MHSA layers and neurons in linear matrices, while depth pruning targets the later transformer blocks by introducing shallow classifiers for early classification. During training, learnable saliency scores are used to determine the importance of both heads and neurons. These saliency scores are continuously updated as part of the learning process. Each head/neuron is assigned a score, and only those with scores above a dynamic threshold are retained. The threshold is also learnable and adjusted throughout training to meet the desired pruning ratio. Model components with scores below the threshold are pruned, reducing model complexity while preserving critical features. For depth pruning, shallow classifiers allow the network to

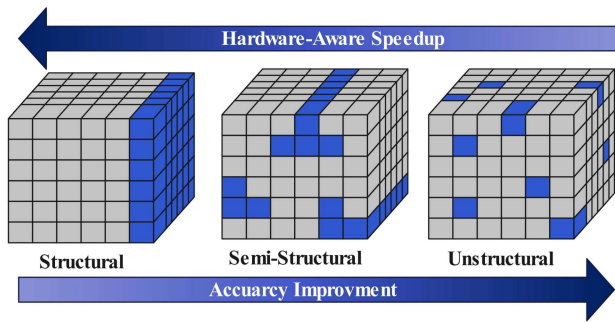


FIGURE 9. Trade-offs of different sparsity schemes in terms of model accuracy and hardware acceleration.

exit early, reducing the number of blocks needed during inference, thus enhancing computational efficiency with minimal accuracy loss.

F. GLOBAL PRUNING

Global pruning involves removing all prunable components across every layer in an integrated approach, considering all levels of pruning.

Huanrui Yang et al. [75] propose a global pruning framework for ViTs aimed at optimizing computational efficiency by pruning and redistributing parameters across transformer blocks. This method targets key components like MSA heads, FFN neurons, and embedding dimensions, addressing the redundancy in these areas. The pruning process occurs during training, guided by a Hessian-aware saliency approach, which assesses the importance of each structural group by analyzing how sensitive the loss function is to changes in the weights of that group. Using this Hessian-based criterion, the framework evaluates the contribution of each group and prunes those with lower importance scores, ensuring minimal impact on model performance. This pruning is performed iteratively, gradually removing less important components while preserving high-priority elements.

Through this process, the framework achieves significant reductions in the number of parameters and FLOPs, leading to much faster inference times while maintaining competitive accuracy. Moreover, the framework incorporates latency-aware regularization, allowing it to directly target reductions in inference time. The result is a more efficient ViT architecture that provides substantial improvements in speed and resource efficiency while preserving accuracy across tasks, making it suitable for deployment in resource-constrained environments.

VI. WHAT ARE THE DIFFERENT GRANULARITIES OF PRUNING IN ViTs?

Pruning techniques in ViTs can be categorized based on their level of granularity into three types: unstructured, semi-structured, and structured. Each approach defines how sparsity is applied within the model, influencing computational efficiency and hardware compatibility. Fig. 9 illustrates the

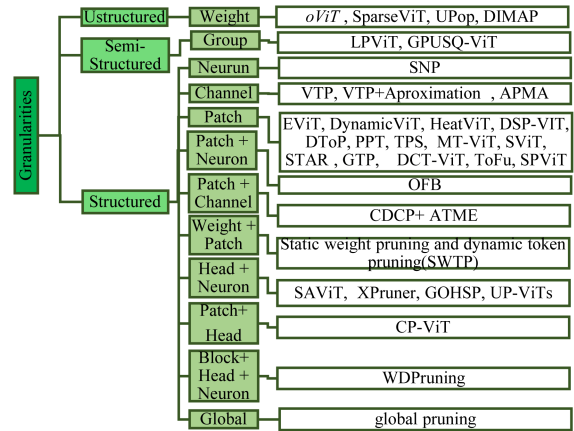


FIGURE 10. An overview of State-of-the-Art ViT Pruning Methods with Different Pruning Levels in Relation to Their Ability to Enhance Processing Speed.

trade-offs between various sparsity methods, highlighting their impact on model accuracy and hardware acceleration. Unstructured pruning involves removing individual weights regardless of their position within the network, reducing the overall number of parameters and thus storage of the model requirements [76]. Semi-structured pruning takes a more organized approach by eliminating groups of weights or connections that follow certain patterns, but without removing entire structures like neurons or channels [77]. This makes it more suitable for standard hardware acceleration. Structured pruning goes further, removing entire structural components such as neurons, channels, heads, patches, or even blocks, significantly reduced computational complexity. This directly translates into speedup during both training and inference phases. Fig. 10 presents a selection of state-of-the-art ViT pruning papers, highlighting various pruning levels and corresponding speedup.

A. UNSTRUCTURED PRUNING

Recent advancements in ViTs have explored unstructured pruning methods such as oViT [40], Sparse-ViT [42], LOST-ViT [43], UPop [44], and DIMAP [45] to reduce memory usage and FLOPs for efficient deployment on resource-constrained devices. However, these approaches often fail to improve inference speed due to the irregular sparsity patterns, which are inefficient on hardware like GPUs and CPUs that are optimized for dense computations. As a result, while unstructured pruning enhances memory efficiency and reduces theoretical complexity, it does not necessarily lead to faster real-world performance, highlighting a gap between computational reduction and actual latency improvement.

B. SEMI-STRUCTURED PRUNING

Kaixin Xu et al. [78] propose LPViT, a low-power, semi-structured pruning technique designed to significantly reduce the computational load and power consumption of ViTs while maintaining their performance in tasks such as image

classification. The method focuses on block-structured pruning, where entire groups of weights within the linear layers, including the query, key, and value matrices (QKV), FFN, and projection layers, are pruned. This approach is more hardware-friendly than unstructured pruning, as it allows for more regular and efficient matrix multiplications that align better with GPU processing capabilities. By using a second-order Taylor approximation to determine which blocks have the least impact on model accuracy, LPViT can achieve a significant reduction in power consumption and computation during inference. The block-wise structure ensures that LPViT can take advantage of GPU kernels optimized for regular data access patterns, resulting in improved inference speed alongside reduced computational demands.

Chong Yu et al. [79] introduce GPUSQ-ViT, a method that specifically targets GPU-friendly optimization through 2:4 structured sparsity and quantization. This approach leverages NVIDIA GPUs support for accelerating sparse matrix operations, enabling a more efficient execution of ViT models. By applying 2:4 structured sparse pruning, GPUSQ-ViT ensures that exactly two out of every four weights are pruned, allowing the remaining structure to be exploited by sparse computation capabilities of the GPU. This targeted sparsity is applied to critical transformer components like Q, K, V projection layers, and feedforward modules. After pruning, the model is further optimized using quantization-aware training (QAT), reducing weights to low-precision formats like INT8 or INT4 while maintaining accuracy through knowledge distillation. This structured approach allows GPUSQ-ViT to maximize the throughput and speedup that modern GPUs can offer, making it ideal for scenarios where real-time inference performance is crucial.

Both LPViT and GPUSQ-ViT illustrate the advantages of semi-structured pruning in leveraging specialized GPU hardware, achieving significant improvements in throughput and inference speed compared to more irregular unstructured pruning methods. By aligning their pruning patterns with the processing strengths of GPUs, these techniques offer a practical solution for deploying ViTs efficiently in performance-critical applications.

C. STRUCTURED PRUNING

Structured pruning methods in ViTs encompass a variety of techniques that simplify the model by removing entire structural components, enhancing their compatibility with hardware acceleration. Neuron pruning methods like SNP [47] focus on eliminating whole neurons from fully connected layers, reducing dense computational requirements. Filter pruning techniques, such as VTP [49], VTP+Approximation [50], and APMA [51], target the removal of filters from convolutional or linear layers, cutting down the number of channels and leading to smaller models and faster inference. Patch pruning methods like EViT [53], DynamicViT [54], and HeatViT [55] remove entire input patches, reducing the sequence length that needs

processing, thus lowering computational loads. Hybrid approaches combine these methods, like OFB [67], which merges patch and neuron pruning, or CDCP+ATME [68], which blends patch and filter pruning, for more balanced optimizations. Head pruning methods such as SAViT [34], X-Pruner [65], and GOHSP [72] remove attention heads that contribute less to the model, simplifying the attention mechanism. CP-ViT [29] integrates both patch and head pruning to cut down on tokens and heads simultaneously. WDPPruning [74] takes a more comprehensive approach, pruning entire blocks, heads, and neurons, thus achieving deeper structural reductions. Lastly, global pruning [75] offers a model-wide perspective by pruning based on global importance scores rather than focusing on any single component, leading to uniform optimization across the model.

All of these structured pruning methods maintain a regular model architecture, making them compatible with hardware accelerators like GPUs. These methods create models that are not only smaller but also faster to run on parallel processing hardware. Hybrid approaches, such as those that combine head and neuron pruning or patch and head pruning, offer a particularly effective balance. They manage to reduce redundant computations in multiple model components simultaneously, which leads to better speedups and lower latency during inference. This holistic optimization makes structured pruning particularly valuable for deploying ViTs, as it ensures that the reductions in model size also translate into improvements in throughput and energy efficiency.

Ultimately, pruning is employed to optimize neural networks by eliminating redundant parameters, resulting in smaller and faster models. Unstructured pruning is particularly effective for memory reduction, while semi-structured pruning strikes a balance between flexibility and speed. Structured pruning, on the other hand, simplifies model architecture to achieve significant speed improvements. Each method aims to optimize the trade-off between computational efficiency and model performance, making pruning an invaluable technique for developing lightweight and efficient deep learning models.

VII. PRUNING SCHEDULES

ViTs are typically pretrained on large datasets to learn general visual patterns and then fine-tuned on smaller, task-specific datasets [78]. Pretraining is essential because training a ViT from scratch requires significant data and computational resources, which is often not feasible for many applications [79]. Fig. 11 illustrates different pruning times for the ViT. ViT pruning can be categorized into three phases: (1) **Pruning during fine-tuning** (also known as *pruning during training*) occurs when the model is being fine-tuned for a specific dataset. This method prunes less important parts of the model during the fine-tuning (training) phase to save resources while maintaining performance. (2) **Post-training pruning** is a popular approach where the model is fully trained and pruning is applied afterward to remove unnecessary parameters, making it efficient for deployment without

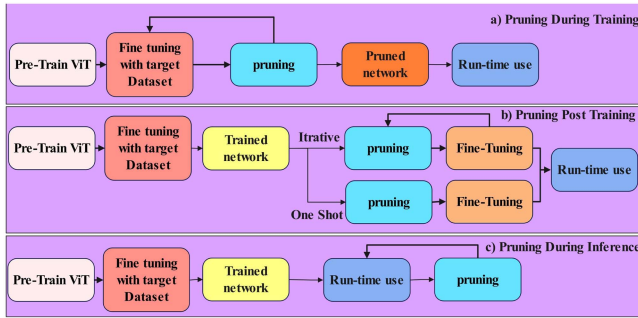


FIGURE 11. Different pruning times for the ViT.

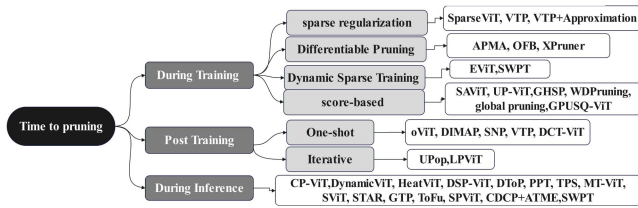


FIGURE 12. Time to prune categorization.

retraining the entire model. This method is widely used in deep learning tasks due to its simplicity and effectiveness. (3) **Dynamic or pruning during inference** adjusts the model based on individual inputs during inference. These methods adaptively prune the model depending on the input data, enabling real-time optimization and reuse of the pruned model for different tasks or inputs, which can lead to efficiency improvements in inference without compromising accuracy. Fig. 12 presents a selection of state-of-the-art ViT pruning papers, highlighting various time to prune.

A. PRUNING DURING FINE-TUNING

In CNNs, pruning during training is a technique used to reduce model complexity and improve efficiency by gradually removing less important weights or structures as training progresses, while still maintaining model performance [80]. This dynamic process allows the network to adaptively focus on the most relevant features, resulting in a more compact and efficient model. Pruning during training in CNNs offers several advantages, such as reduced computational cost [81], lower memory requirements [82], and faster inference times, making models more suitable for deployment on resource-constrained devices. Additionally, it can improve generalization by preventing overfitting through the removal of less significant parameters [83]. However, pruning introduces challenges, including the potential removal of weights that may become important later in training, possibly leading to suboptimal performance, and adds complexity to the training process by necessitating careful tuning of pruning schedules and thresholds. In contrast, ViTs are typically first pre-trained on large-scale datasets due to their lack of inherent inductive biases such as translation invariance and local receptive

fields, which makes them more reliant on large datasets for learning robust representations [84], [85]. Following pre-training, pruning is generally applied during the fine-tuning phase to adapt the model to smaller or domain-specific datasets. This targeted pruning helps remove less relevant parameters, thereby improving model efficiency and performance for task-specific applications while also reducing overfitting and optimizing computational resources.

The process of pruning during fine-tuning typically begins with a pre-trained ViT, represented as $f(x; W_0)$, where W_0 represents the initial weights. During fine-tuning, both the weights W and the pruning masks M are updated simultaneously, allowing the model to selectively remove unnecessary components such as layers, attention heads, or image patches. Similar to CNNs, this paper categorizes the main approaches to pruning in ViTs into four paradigms: (1) sparsity regularization, (2) dynamic sparse training, (3) score-based pruning, and (4) differentiable pruning.

1) SPARSITY REGULARIZATION

Sparsity regularization is a widely adopted technique in pruning during training approaches [86]. Sparsity regularization encourages the network weights to shrink toward zero, promoting a sparse representation [87]. Concurrently, pruning systematically eliminates these small or near-zero weights, as well as potentially entire neurons or filters, based on predefined criteria that assess their contribution to the network performance. By progressively reducing the network complexity during training, this approach allows the model to adapt to a streamlined structure, thus maintaining or even enhancing accuracy while significantly improving computational efficiency and memory utilization. The resulting network remains both sparse and optimized, achieving a balance between model compactness and predictive capability. Examples of this technique can be found in works such as [42], [49], and [50].

2) DYNAMIC SPARSE TRAINING

Dynamic sparse training is a technique used to optimize model by continuously adjusting the sparse structure of the network during training [88]. The method involves periodically identifying and pruning a fraction of the least important weights, based on some criteria such as their magnitude or contribution to the loss function [89]. After pruning, an equal number of weights are regrown, often by randomly initializing new connections or based on specific heuristics that predict which new connections might be beneficial [90]. This iterative prune-and-grow cycle helps the network adaptively refine its architecture to find more effective sparse configurations, potentially leading to improved performance with reduced computational cost. Examples of this technique can be found in works such as [53] and [69].

3) SCORE-BASED PRUNING

Pruning during training often relies on scoring criteria to determine which connections or neurons to remove [91]. As the network trains, a score is assigned to each connection based on metrics such as magnitude, gradients, or other criteria that reflect their contribution to the overall performance. Weights with the lowest scores are pruned. This process is repeated iteratively throughout training, allowing the network to adapt to the reduced structure while maintaining or improving its performance. Score-based pruning helps achieve a balance between model efficiency and accuracy, making the network more compact and computationally efficient. Examples of this technique can be found in works such as [34], [37], [72], [74], [75], and [92].

4) DIFFERENTIABLE PRUNING

Differentiable network compression has gained traction as a powerful approach to pruning model by leveraging gradient-based optimization techniques [93]. These methods incorporate pruning into the training process by introducing differentiable parameters that control which weights or neurons are retained. For instance, learnable scaling factors or binary masks can be applied to weights, enabling the network to adjust its structure in a continuous and differentiable manner. The gradients of these parameters are computed alongside the standard weight gradients, allowing pruning decisions to be optimized using backpropagation. By integrating pruning with the training process, differentiable approaches can dynamically discover efficient network architectures, improving model compactness without compromising accuracy. Examples of this technique can be found in works such as [51], [67] and [71].

B. POST TRAINING PRUNING

Post-training pruning has emerged as a widely adopted approach within CNNs, often considered the standard paradigm for model compression. A primary advantage of post-training pruning lies in its simplicity, as it avoids the need for alterations to the training process, thereby facilitating easier implementation [94]. This approach enables the model to reach its full capacity before pruning less significant parameters, helping preserve initial performance. However, since pruning is not integrated into the training process, post-training pruning may be less efficient than methods applied during training [80]. Consequently, this can lead to suboptimal trade-offs between model sparsity and performance.

When applied to ViTs, the post-training pruning procedure follows a multi-step process:

- 1) *Initial Training*: The network, initially a randomly initialized dense model $f(x; W_0)$, is pre-trained to convergence, reaching a state $f(x; W_t)$
- 2) *Post-training Pruning*: The pruning stage involves identifying and removing weights (or other components like attention heads or patches) that contribute the least to overall performance of the model. The pruned network

$f(x; W_t' \odot M')$, where W_t' and M' are the weights and pruning masks after pruning.

- 3) *Re-training or Fine-tuning*: Finally, the remaining weights are either re-trained from scratch $f(x; W_0 \odot M'')$ or fine-tuned $f(x; W_t'' \odot M'')$ to fully recover performance of the network, with W_t'' and M'' denoting the final weights and masks after pruning. During this process, the sparsity level can be incrementally increased until the target sparsity is reached.

Pruning and fine-tuning may be carried out once (one-shot pruning) or multiple times (iterative pruning) to gradually refine the model. Notable examples of one-shot pruning include studies such as [40], [45], [47], [56], and [64], while iterative pruning is demonstrated in works like [44] and [95].

C. DURING INFERENCE PRUNING

Dynamic pruning during inference, or runtime pruning, allows neural networks to adapt their architecture based on individual inputs rather than relying on a fixed, statically pruned model [96]. This technique is grounded in the idea that different inputs require varying levels of computational effort, as the complexity of generating accurate predictions is not uniform across tasks [97]. By adjusting the structure of network dynamically according to input demands, inference can be made more efficient [98]. The main advantage of this approach is its ability to significantly reduce computational costs without compromising accuracy, particularly for simpler inputs that do not require the full model capacity [99]. However, the need to reconfigure the architecture for each input introduces overhead during inference, which may reduce some of the efficiency gains, especially for tasks involving frequent and complex adjustments.

In the case of ViTs, dynamic pruning is particularly beneficial due to intrinsic of the architecture reliance on input-dependent attention mechanisms. Unlike CNNs, where feature representations are static, ViTs dynamically calculate attention scores for each input, resulting in varying importance across different patches [100]. Consequently, dynamic pruning techniques, especially for patch pruning, are widely used in ViTs, as they can effectively exploit the variability in patch importance for different inputs, resulting in a more efficient model. Research works such as [29], [54], [55], [56], [57], [58], [59], [60], [61], [62], [63], [65], [66], [68], and [69] have explored various dynamic pruning strategies in ViTs, demonstrating capability of the method to optimize performance and computational efficiency by pruning patches or attention heads in an input-adaptive manner.

VIII. PRUNING CRITERIA

Pruning criteria in ViTs can be broadly categorized into several types, including l_p Norm, loss change, relevance-based, sensitivity and/or saliency, attention score-based, and prediction module methods. While these categories highlight distinct aspects of pruning, the boundaries between them are often flexible, with each method emphasizing different aspects of model efficiency and accuracy. Fig. 13 presents a selection

Pruning criteria	l_p Norm	SparseViT, VTP, VTP+Aproximation, SWTP
	Loss Change	UP-ViT, DIMAP, ToFu, LPViT, GPUSQ-ViT
	Explainability-driven	STAR, XPruner
	Sensitivity / Saliency	SAViT, oViT, UPop, APMA, DCT-ViT, CDCP+ATME, WDPPruning, global pruning
	Attention Scores	CP-ViT, SNP, EViT, HeatViT, PPT, TPS, SRAT, GTP, SPViT, OFB, SWTP, GOHSP
	Predictive Module	DynamicViT, HeatViT, DSP-ViT, DToP, MT-ViT, SViT, CDCP+ATME

FIGURE 13. Pruning criteria categorization.

of state-of-the-art ViT pruning papers, highlighting various pruning criteria.

A. l_p NORM

In pruning methods with l_p Norm criterion, elements with lower norms are removed, under the assumption that smaller magnitudes contribute less to model performance. Its efficiency and simplicity make it a widely used approach; however, it may overlook important dependencies, as low-norm weights can still contribute significantly in complex architectures. Several works have demonstrated the use of l_p Norm pruning in ViTs [42], [49], [50], [69].

B. LOSS CHANGE

The loss change criterion for pruning assesses the importance of specific weights or components by measuring the resulting change in loss upon their removal, aiming to identify elements whose elimination has minimal impact on overall model performance. This approach directly assesses the importance of weights in preserving the loss landscape, making it particularly effective in determining which parameters can be pruned without major degradation in accuracy. Its advantage lies in its more nuanced pruning decisions, potentially retaining important dependencies that simpler criteria might overlook, which is critical for complex models like ViTs. However, it can be computationally intensive, especially for large networks. Several studies on ViTs ([37], [45], [65], [95], [92]) have applied loss change criteria to maintain accuracy during pruning.

C. EXPLAINABILITY-DRIVEN SCORE

Pruning ViT can be guided by criteria based on Explainable AI (XAI) techniques. XAI methods use interpretability tools like attention maps or saliency maps to rank the importance of different units within the model, identifying less significant components that can be pruned. LRP, an XAI technique, is a backward propagation technique that calculates relevance scores for each unit by distributing output of the model back through the layers. This enables the identification of units that have the most significant impact on the classification outcome. While LRP-based pruning is advantageous for its interpretability, providing a clear rationale for which units are retained or discarded, it can be computationally intensive and

complex due to the need for gradient calculations and score normalization.

In [71], LRP is applied to ViT models by retaining patches with high relevance scores, thereby improving computational efficiency while maintaining accuracy. In [62], LRP is used to evaluate the importance of each patch in the ViT model, using the LRP equation as:

$$R_l^j = \sum_i x_l^j \frac{\partial L_l(X_l, W_l)}{\partial x_l^j} \frac{R_{l+1}^i}{L_l(X_l, W_l)} \quad (11)$$

where R_l^j is the relevance score assigned to patch j at layer l , showing its contribution to the output, x_l^j represents the input activation for that layer, the gradient $\frac{\partial L_l(X_l, W_l)}{\partial x_l^j}$ showing how changes in this patch influence the output, R_{l+1}^i is the relevance score from the next layer $l+1$, distributed back to layer l based on the contribution of each patch, and lastly, $L_l(X_l, W_l)$ normalizes relevance distribution, ensuring scores are proportionally allocated across patches. This scoring framework allows for selective pruning in ViTs by retaining only the most relevant patches across layers, which reduces computational load while maintaining model performance.

D. SENSITIVITY AND/OR SALIENCY

Sensitivity-based pruning focuses on the magnitude of the gradient of the loss function with respect to each weight [101], while saliency-based methods extend this by considering the second-order derivative (Hessian) to capture correlations between weights [102]. These criteria are advantageous as they provide a more precise indication of weight importance by accounting for both impact of the weight on the loss and potential correlations with other weights, enabling a more fine-tuned pruning strategy. However, calculating these criteria can be computationally intensive, especially for large models, due to the need for gradient and sometimes Hessian information, which can increase the overall pruning complexity [103].

In ViTs, sensitivity and saliency criteria have been explored for pruning in several works, such as [40], [44], [51], [64], [68], [74] and [75] to maintain accuracy while reducing model size. For instance, in [34], an importance measure I_i is defined as:

$$I_i = \frac{1}{N} \sum_{n=1}^N w_i^2 \frac{\partial^2 L_n}{\partial w_i^2} = \frac{1}{N} \sum_{n=1}^N \left(w_i \frac{\partial L_n}{\partial w_i} \right)^2 \quad (12)$$

where N is the total number of samples, w_i represents the weight being evaluated, L_n is the loss for the n -th data sample, and $\frac{\partial L_n}{\partial w_i}$ and $\frac{\partial^2 L_n}{\partial w_i^2}$ are the first and second derivatives of the loss with respect to w_i . By using this diagonal Hessian approximation, the method estimates the sensitivity of the loss to each weight, enabling informed pruning decisions that consider the model's structural dependencies.

E. ATTENTION SCORES

The attention score criterion leverages the attention mechanism to identify important information within input sequences, making it an effective tool for pruning ViTs. Attention scores represent the relevance of each token (or patch), in the case of ViTs to other tokens in the sequence, making them a natural choice for determining which parts of the input contribute most to model predictions [104]. This criterion offers the advantage of retaining contextually significant information while discarding redundant or low-impact tokens, thereby reducing model complexity without substantial accuracy loss. A limitation, however, is that attention scores alone may not capture deeper long-range dependencies, leading to the potential loss of subtle yet crucial information [62].

The attention score criterion is frequently applied in dynamic or inference-time pruning, especially at the patch level in ViTs, as patches often vary in importance based on content. This approach allows for a more adaptable model that can adjust to the relevance of different patches, as demonstrated in studies such as [29], [66], [59], [62], [63], [55], [47], [67], [69], [72], and [53]. For instance, in [58], an attention-based importance score for the i -th patch is defined as:

$$Score_i = \frac{A_{1,i+1} \times ||V_{i+1}||}{\sum_{j=1}^N A_{1,j+1} \times ||V_{j+1}||} \quad (13)$$

where $A_{1,i+1}$ represents attention scores between the class token and image tokens, and V_{i+1} represents the value matrix from the transformer's attention mechanism. Tokens with lower scores are pruned using a top-K selection policy, while highly similar tokens are pooled.

F. PREDICTIVE MODULE

In ViTs, dynamic pruning is critical due to its potential impact on final output quality. One effective approach to evaluate this impact is by employing a predictive model, such as a MLP or CNN. These models are pre-trained to estimate the importance of specific elements in the output, subsequently guiding pruning decisions during inference. This predictive mechanism allows for selective pruning, retaining elements that contribute meaningfully to the final result. Several studies, including, [54], [55], [56], [57], [60], and [68], have applied this technique to enhance ViT pruning efficiency. For instance, [61] demonstrates this approach by using a two-layer MLP to evaluate and prune tokens, achieving improved computational efficiency while preserving accuracy.

IX. QUANTITATIVE COMPARISON OF VIT PRUNING METHODS

This section has presented a comprehensive analysis of ViT pruning methods for classification on the ImageNet-1K dataset, as reported across recent papers and summarized in Section V. The comparison has encompassed critical performance indicators, including the total number of parameters, percentage of parameter reduction, FLOPs, FLOPs reduction,

TABLE 2. Quantitative Comparison of Pruned Models Based on DeiT-B on the ImageNet-1K Classification Task. The Best Results are in Bold. (–) Indicates Unreported Data.

model	#Par	Para. ↓(%)	FLOP s	FLOP s ↓ (%)	Top1 acc.(%)	Top1 acc↓ (%)	Memor y (MB)	Throughput(i m/s)
VTP (40% pruned)	48	44.83	10	43.18	80.7	2.7	183.1	-
WDPPruning(20%, depth(2))	59.4	31.72	10.8	38.64	80.85	2.55	226.5	696
WDPPruning(30%, depth(1))	55.3	36.44	9.9	43.75	80.76	2.64	211.0	707
XPruner	-	-	8.5	51.70	81.2	2.2	-	-
DPS-ViT	-	-	9.4	46.59	81.6	1.8	-	413
SAViT	33.6	61.38	6.7	61.93	83.31	0.09	128.3	-
SNP	31.6	63.68	6.4	63.64	79.63	3.77	120.6	-
CP-ViT	-	-	13.69	22.22	80.91	2.49	-	-
GPUSQ(INT8)	13.55	84.43	0.55	96.88	82.9	0.5	12.9	-
GPUSQ(INT4)	6.78	92.21	0.28	98.41	81.6	1.8	3.2	-
LPViT	-	-	5.1	71.02	81.03	2.37	-	-
STAR(R=0.7)	-	-	9.3	47.16	81	2.4	-	1944.5
OFB	43.9	49.54	8.7	50.57	81.7	1.7	167.5	-
APMP	87	0.00	6.2	64.77	83.19	0.21	332.7	-
MT-ViT	87	0.00	13.9	21.02	81.8	1.6	332.7	-
CAIT	71.3	18.05	7.4	57.95	81.6	1.8	271.9	1206
oViT(90% pruned)	-	-	2.21	87.44	79.7	3.7	-	-
global pruning	17	80.46	6.8	61.36	83.29	0.11	64.8	-

Top-1 accuracy, and the corresponding accuracy degradation after pruning.

Table 2 provides a quantitative comparison of various pruned models based on the DeiT-B architecture. Among all evaluated methods, the GPUSQ pruning approach [92] achieves the lowest FLOPs scores in both int8 and int4 settings, with values of 0.55 and 0.28, respectively. However, GPUSQ incorporates quantization alongside pruning, which significantly contributes to its enhanced performance. Therefore, its efficiency cannot be solely attributed to pruning. When focusing on models that rely exclusively on pruning, the oViT(90% pruned) model [40] achieves the lowest FLOPs value of 2.21, indicating that it is the most efficient among the pruning-only methods. In scenarios where preserving high accuracy is critical and minimal performance degradation is acceptable, SAViT [34] and Global Pruning [75] emerge as the most suitable options. These models exhibit relatively low accuracy drops of 0.09% and 0.11%, respectively, while still providing meaningful reductions in computational cost.

Table 3 presents a comprehensive evaluation of various pruned models based on the DeiT-S architecture. Among all evaluated approaches, similar to the results observed with DeiT-B, GPUSQ achieves the lowest FLOPs, attaining FLOPs of 0.14 in the INT8 setting and 0.07 in INT4. However, these results are due to the combined use of pruning and quantization, with quantization significantly contributing to the improved efficiency. When considering methods that use pruning alone, oViT (90% pruned) and OFB achieve the lowest FLOPs, with 0.73 and 0.9 respectively. In scenarios where preserving model accuracy is critical and only minimal degradation is acceptable, SAViT emerges as the most favorable choice. It maintains an accuracy of 83.31% while still providing substantial computational savings.

In summary, results across DeiT-B and DeiT-S consistently demonstrate that GPUSQ achieves the lowest FLOPs,

TABLE 3. Quantitative Comparison of Pruned Models Based on DeiT-S on the ImageNet-1K Classification Task. The Best Results are in Bold. (–) Indicates Unreported Data.

model	#Par	Para. ↓(%)	FLOP s	FLOP s ↓ (%)	Top 1 acc. (%)	Top1 acc.↓ (%)	Memory (MB)	Throughput (im/s)
WDPruning(30%,depth(1))	13.3	37.6	2.6	43.48	78.38	2.82	50.7	1830
X-Pruner	-	-	2.4	47.83	78.93	2.27	-	-
DPS-ViT	-	-	2.4	47.83	79.5	1.7	-	1342
UP-ViT	22.1	0	3.2	30.43	81.56	-0.36	84.3	603.1
SAViT	20.6	6.78	4.2	8.70	83.31	-2.11	78.6	-
SNP	10	54.75	2	56.52	78.52	2.68	38.1	-
CP-ViT	-	-	2.65	42.20	79.08	2.12	-	-
OPTIN	-	-	3.15	31.52	79.24	1.96	-	-
OHSP(50% pruned)	11.1	23.97	2.80	39.00	79.86	1.34	42.3	-
GPUSQ(INT8)	3.46	84.3	0.14	96.96	80.3	0.9	3.3	-
GPUSQ(INT4)	1.73	92.15	0.07	98.48	79.3	1.9	0.8	-
LPViT	-	-	2.3	50.00	80.69	0.51	-	-
UPOP(50% pruned)	11.4	48	2.3	50	78.9	2.3	43.5	-
STAR(R=0.7)	-	-	2.3	50.00	79.2	2	-	4080.6
OFB	4.4	80	0.9	80.43	75	6.2	16.8	-
TOFU	-	-	2.7	41.30	79.6	1.6	-	-
APMP	-	-	4.2	8.70	82.35	-1.15	-	-
MT-ViT	-	-	2.5	45.65	79.5	1.7	-	-
TPS	22.05	0	3	34.78	79.7	1.5	84.1	-
EViT(50% pruned)	-	-	2.3	50.00	78.5	2.7	-	5408
CAIT	18.4	16.74	2.1	54.35	80.2	1	70.2	2968
oViT(90% pruned)	-	-	0.73	84.1	75.2	6	-	-
global pruning	10.5	52.27	4.2	8.70	82.19	-0.99	40	-

primarily due to the combined effects of pruning and quantization. While these findings highlight the effectiveness of hybrid compression techniques, they also emphasize the importance of evaluating pruning strategies independently to ensure fair comparisons. Among pruning-only methods, oViT models consistently deliver strong compression. Additionally, for scenarios where preserving model accuracy is a top priority, methods such as SAViT and Global Pruning offer compelling trade-offs, minimizing accuracy degradation while still achieving substantial reductions in computational complexity. These insights provide practical guidance for selecting pruning strategies based on specific deployment requirements, such as accuracy sensitivity or hardware constraints.

X. SUGGESTION AND FUTURE DIRECTION

Different pruning methods are discussed in this section and promising directions for future work are provided.

A. SUGGESTIONS ON PRUNING METHOD SELECTION

Given that ViTs are relatively new, compression and optimization strategies for these models remain underexplored, leaving significant room for further research. Existing studies lack a universally accepted standard for determining the most effective pruning approach, as different methods are suited to distinct application requirements and varying hardware or software resources. Accordingly, there is a need to both refine current pruning techniques and introduce more efficient methods tailored to the unique architecture of ViTs. The following sections offer general recommendations for selecting suitable pruning strategies for developing advanced pruning methods.

- 1) Dynamic pruning is better suited for patch pruning, as it allows input-specific selection of patches, enabling the model to focus on relevant regions and improve performance. In contrast, static pruning is preferable at other levels due to its consistent inference times and memory compression, making it ideal for latency-sensitive scenarios.
- 2) For optimal speedup, unstructured pruning is preferred with specialized hardware like FPGAs or ASICs, which can efficiently process sparse matrices. In contrast, structured pruning is more suitable for standard hardware such as CPUs or GPUs, as it preserves regular patterns that align with conventional architectures, enabling efficient computation without specialized support
- 3) In pruning, both static and dynamic scoring are valuable depending on the objective. Static scoring evaluates component importance across all inputs, identifying elements that can be pruned with minimal impact on overall performance. In contrast, dynamic scoring assesses importance per input, enabling adaptive pruning tailored to input-specific characteristics. This enhances model flexibility, especially when input variability is high.
- 4) In dynamic pruning, if performance is highly important and there are no constraints on computational or memory resources, a modular predictor is preferred because it can accurately predict the effect of pruning on the final result, allowing for precise adaptation to input variability. However, when memory and computational capacity are limited, an attention-based criterion is more suitable. This approach dynamically adjusts pruning by focusing on the most relevant parts of each input, optimizing resource usage while maintaining adaptability.
- 5) When sufficient computational resources are available during training pruning, iterative pruning-after-training methods are recommended, as they effectively minimize performance degradation at a given pruning ratio. Conversely, if resources are limited for both pruning and inference, one-shot post-training pruning methods offer a more efficient alternative.
- 6) When performance is the top priority, loss change-based pruning criteria are generally the best choice, as they directly aim to minimize the accuracy degradation during pruning by evaluating the impact of each element on loss of the model. This approach ensures that the most important parameters for performance are retained, making it ideal when maintaining high accuracy is crucial. However, these methods can be computationally expensive due to the need for back-propagation and gradient calculations. Alternatively, sensitivity or saliency-based pruning can be effective in tasks where retaining the most crucial parts of the network for specific data or tasks is important, offering a more data-driven and adaptive approach to pruning.

- 7) When the pruning time is critical, l_p Norm criterion are typically the best choice. These methods are computationally efficient because they focus on pruning based on the magnitude of the elements, requiring minimal additional calculations. They can quickly identify and remove less important parameters without needing expensive forward or backward passes through the network. While they may not always perfectly preserve accuracy, they offer a good trade-off between speed and performance, especially in resource-constrained environments.
- 8) The attention score criterion is the best choice for pruning during inference time because it preserves the importance of patches, thereby maintaining accuracy and performance. By focusing on the attention scores, the pruning process adapts to each individual input, ensuring that only the most relevant parts of the model are retained for each specific task. This dynamic pruning approach makes it highly effective for optimizing model efficiency during inference. However, for static pruning, attention scores may not be the optimal criterion, as they are computed based on each input, and static pruning typically requires a more general, fixed pruning strategy that does not adapt to individual inputs.
- 9) Pruning approaches should be specifically designed for ViTs, considering attention heads, tokens, and transformer blocks rather than borrowing strategies from the CNN side.
- 10) Lightweight pruning strategies may be suitable for real-time or edge operation, while aggressive pruning is best left to scenarios where memory or energy constraints are of critical importance.

B. FUTURE DIRECTIONS

Future research should aim to address several key aspects of ViTs.

- *Component-wise Sensitivity Analysis for Interpretable Pruning*: Due to the heterogeneous architecture of ViT, where each layer and component within a transformer block serves a distinct role, it is essential to understand the sensitivity and contribution of each part to the overall model performance. Unlike CNNs, ViT cannot be effectively pruned using uniform strategies across all layers. Future work should focus on detailed sensitivity analysis of individual components such as tokens, attention heads, and feed-forward layers to assess their relative importance and susceptibility to pruning. This analysis would support a transition from treating ViT as opaque black boxes to a more interpretable white box approach. Enhancing interpretability in this way would provide valuable insights into which components are redundant and why, leading to more informed and effective pruning strategies.
- *Reevaluating ViT Architecture Through Pruning*: An important direction for future research is to examine

whether certain redundant components within ViT may actually impair model performance. Since the ViT architecture is directly adapted from the transformer model originally designed for language tasks, it may not be optimally suited for visual data. This raises the hypothesis that pruning is not only effective for reducing model size and computational cost, but could also improve accuracy by removing unnecessary or noisy components. Investigating this possibility may lead to the development of more tailored and efficient ViT architectures, better aligned with the characteristics of image-based tasks. Such efforts would enhance our understanding of how model structure influences generalization, supporting pruning strategies that contribute to both compression and improved performance.

- *Emerging Trends in ViT Pruning*: Future research on pruning for ViTs should focus on designing algorithms that learn network architectures in a more economical, efficient, and effective manner. ViT pruning is also increasingly being integrated with advanced learning frameworks, including Automated Machine Learning [105], Neural Architecture Search [106], continual learning [107], lifelong learning [108], federated learning [109] and contrastive learning [110]. As network energy demands increase, energy-aware pruning is becoming increasingly important. While early efforts have focused on reducing computational and memory costs, these do not always lead to proportional energy savings. An emerging direction involves integrating pruning methods directly into hardware, enabling efficient deployment of pruned ViTs on platforms optimized for such architectures, such as FPGA-based systems.
- *Task-Specific Pruning Strategies for ViTs*: ViTs excel in complex tasks such as object detection, classification, and segmentation due to their global attention mechanism. However, their uniform processing of all tokens and high computational cost makes them inefficient for real-time or resource-constrained deployment, and many components contribute unevenly to task performance. Pruning is therefore necessary to remove redundant tokens, attention heads, or feed-forward units while preserving task-relevant information. Future research should explore task-specific pruning strategies tailored to the requirements of each application. For object detection, pruning should retain spatially critical patches and fine-tune positional embeddings to preserve localization and multi-scale features. For classification, more aggressive pruning of redundant patches can focus on global feature retention. In segmentation, pruning criteria should emphasize patch saliency and context-preserving attention heads to maintain pixel-level accuracy. Additionally, multi-level pruning, where different layers are pruned according to their contribution, combined with dynamic, adaptive criteria can optimize efficiency while minimizing accuracy loss. These approaches will enable ViTs to be deployed effectively

in complex vision tasks with improved computational efficiency.

- *ViT Pruning for Image Fusion Applications:* Modern image fusion networks increasingly adopt ViTs to capture long-range and cross-modal dependencies; however, their high computational cost remains a major limitation for practical deployment. Recent studies demonstrate that token pruning and token fusion are particularly effective for improving efficiency while preserving fusion quality. For instance, hardware–software co-design approaches such as TAFP-ViT [111] illustrate that combining adaptive token pruning with fused QKV operations can significantly reduce memory and latency, making ViT-based fusion more viable for embedded systems. More recently, attention-guided and graph-based pruning strategies, exemplified by TRAM [112], shows that token importance can be estimated without retraining, enabling aggressive pruning while maintaining accuracy. These advances suggest future research focus on fusion-aware pruning strategies that jointly consider token importance, cross-modal alignment, and hardware efficiency. Developing pruning criteria tailored to multi-modal fusion tasks, such as medical or multi-sensor image fusion, offers a promising path toward lightweight, high-quality ViT-based fusion models.
- *Efficient ViT Pruning for Moving Object Detection:* ViT-based models for moving object and video detection face high computational costs that limit real-time deployment. Recent studies show that exploiting temporal redundancy through adaptive pruning can significantly improve efficiency while preserving accuracy. For example, RAAG [113] introduces redundancy-adaptive and attention-guided token pruning, dynamically adjusting pruning rates across frames and transformer layers to balance efficiency and detection performance. Similarly, MaskVD [114] leverages temporal feature reuse to mask redundant regions across consecutive frames, effectively pruning input tokens with minimal accuracy loss. These methods demonstrate that motion-aware and temporally adaptive pruning is particularly effective for dynamic scenes. Future research should focus on integrating temporal consistency and motion cues into pruning strategies to enable efficient ViT deployment for real-time moving object detection.

XI. CONCLUSION

This paper presents a comprehensive analysis of pruning techniques specifically designed for ViTs, emphasizing the key distinctions between pruning in ViTs and CNNs. Due to the unique architectural characteristics of ViTs such as their structural design, feature extraction mechanisms, and input dependency, many traditional CNN pruning methods are not directly applicable. To address these differences, a structured framework is introduced, categorizing ViT pruning approaches based on four main aspects: pruning components, pruning granularity, pruning schedules, and pruning criteria

Following the framework, a quantitative comparison of recent ViT pruning methods is provided, evaluating Top-1 accuracy, FLOPs, and parameter counts to highlight performance-efficiency trade-offs. In addition, practical guidelines are outlined to assist in selecting pruning strategies that align with specific model objectives and resource constraints. Key directions for future research are identified, including component-wise sensitivity analysis for interpretable pruning, reevaluation of ViT architecture through pruning, exploration of task-specific strategies, and investigation of emerging trends in the field. These advancements aim to facilitate the development of more efficient, interpretable, and deployment-ready ViT models. As ViTs continue to expand across a wide range of vision tasks, advanced and adaptive pruning techniques will be essential to ensure their scalability and applicability in real-world environments.

REFERENCES

- [1] N. Q. K. Le, “Leveraging transformers-based language models in proteome bioinformatics,” *Proteomics*, vol. 23, no. 23–24, Dec. 2023, Art. no. e2300011.
- [2] Q. Fournier, G. M. Caron, and D. Aloise, “A practical survey on faster and lighter transformers,” *ACM Comput. Surv.*, vol. 55, no. 14, 2023, Art. no. 304, doi: [10.1145/3586074](https://doi.org/10.1145/3586074).
- [3] H. Dong, L. Zhang, and B. Zou, “Exploring vision transformers for polarimetric SAR image classification,” *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5219715, doi: [10.1109/TGRS.2021.3137383](https://doi.org/10.1109/TGRS.2021.3137383).
- [4] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. 9th Int. Conf. Learn. Representations*, 2021, pp. 1–4.
- [5] H. Cao, Z. Qu, G. Chen, X. Li, L. Thiele, and A. Knoll, “GhostViT: Expediting vision transformers via cheap operations,” *IEEE Trans. Artif. Intell.*, vol. 5, no. 6, pp. 2517–2525, Jun. 2024, doi: [10.1109/TAI.2023.3326795](https://doi.org/10.1109/TAI.2023.3326795).
- [6] L. Papa, P. Russo, I. Amerini, and L. Zhou, “A survey on efficient vision transformers: Algorithms, techniques, and performance benchmarking,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 7682–7700, Dec. 2024.
- [7] Z. Lu, C. Ding, F. Juefei-Xu, V. N. Boddeti, S. Wang, and Y. Yang, “TFormer: A transmission-friendly ViT model for IoT devices,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 2, pp. 598–610, Feb. 2023, doi: [10.1109/TPDS.2022.3222765](https://doi.org/10.1109/TPDS.2022.3222765).
- [8] C. Chen, H. Min, Y. Peng, Y. Yang, and Z. Wang, “An intelligent real-time object detection system on drones,” *Appl. Sci.*, vol. 12, no. 20, 2022, Art. no. 10227, doi: [10.3390/app122010227](https://doi.org/10.3390/app122010227).
- [9] G. Xu, Z. Hao, Y. Luo, H. Hu, J. An, and S. Mao, “DeViT: Decomposing vision transformers for collaborative inference in edge devices,” *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 5917–5932, May 2024, doi: [10.1109/TMC.2023.3315138](https://doi.org/10.1109/TMC.2023.3315138).
- [10] L. Papa, P. Russo, and I. Amerini, “METER: A mobile vision transformer architecture for monocular depth estimation,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 10, pp. 5882–5893, Oct. 2023, doi: [10.1109/TCSVT.2023.3260310](https://doi.org/10.1109/TCSVT.2023.3260310).
- [11] J. Lee, Y. Kwon, S. Park, M. Yu, J. Park, and H. Song, “Q-HyViT: Post-training quantization of hybrid vision transformers with bridge block reconstruction for IoT systems,” *IEEE Internet Things J.*, vol. 11, no. 22, pp. 36384–36396, Nov. 2024.
- [12] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, “Linformer: Self-attention with linear complexity,” 2020, *arXiv:2006.04768*.
- [13] Z. Liu et al., “Swin transformer: Hierarchical vision transformer using shifted windows,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 10012–10022.
- [14] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training data-efficient image transformers & distillation through attention,” in *Proc. Mach. Learn. Res.*, 2021.
- [15] G. Hinton, “Distilling the knowledge in a neural network,” 2015, *arXiv:1503.02531*.

- [16] M. H. Javani, M. Zandehshahvar, and A. Adibi, "Reducing computation complexity in optimization of nanophotonic structures using pruning," *IEEE Photon. Technol. Lett.*, vol. 36, no. 4, pp. 215–218, Feb. 2024, doi: [10.1109/LPT.2023.3342631](https://doi.org/10.1109/LPT.2023.3342631).
- [17] H. Cheng, M. Zhang, and J. Q. Shi, "A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10558–10578, Dec. 2024.
- [18] Y. He and L. Xiao, "Structured pruning for deep convolutional neural networks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 5, pp. 2900–2919, May 2024, doi: [10.1109/TPAMI.2023.3334614](https://doi.org/10.1109/TPAMI.2023.3334614).
- [19] T. Liang, J. Glossner, L. Wang, S. Shi, and X. Zhang, "Pruning and quantization for deep neural network acceleration: A survey," *Neurocomputing*, vol. 461, 2021, pp. 370–403, doi: [10.1016/j.neucom.2021.07.045](https://doi.org/10.1016/j.neucom.2021.07.045).
- [20] S. Huang, E. Tang, S. Li, X. Ping, and R. Chen, "Hardware-friendly compression and hardware acceleration for transformer: A survey," *Electron. Res. Arch.*, vol. 30, no. 10, 2022, doi: [10.3934/era.2022192](https://doi.org/10.3934/era.2022192).
- [21] K. T. Chitty-Venkata, S. Mittal, M. Emani, V. Vishwanath, and A. K. Somani, "A survey of techniques for optimizing transformer inference," *J. Syst. Archit.*, vol. 144, 2023, Art. no. 102990.
- [22] Y. Tang et al., "A survey on transformer compression," 2024, *arXiv:2402.05964*.
- [23] B. J. Kang et al., "A survey of FPGA and ASIC designs for transformer inference acceleration and optimization," *J. Syst. Archit.*, vol. 155, 2024, Art. no. 103247, doi: [10.1016/j.sysarc.2024.103247](https://doi.org/10.1016/j.sysarc.2024.103247).
- [24] G. Habib, T. J. Saleem, and B. Lall, "Knowledge distillation in vision transformers: A critical review," 2023, *arXiv:2302.02108*.
- [25] D. Du, G. Gong, and X. Chu, "Model quantization and hardware acceleration for vision transformers: A comprehensive survey," 2024, *arXiv:2405.00314*.
- [26] M. Chen, J. Gao, and W. Yu, "Lightweight and optimization acceleration methods for vision transformer: A review," in *IEEE Conf. Intell. Transp. Syst.*, 2022, pp. 2154–2160, doi: [10.1109/ITSC55140.2022.9921989](https://doi.org/10.1109/ITSC55140.2022.9921989).
- [27] F. Chen, Z. Luo, L. Zhou, X. Pan, and Y. Jiang, "Comprehensive survey of model compression and speed up for vision transformers," *J. Inform. Technol. Policy*, pp. 1–12, Apr. 2024.
- [28] Z. Zhang and X. Gong, "Positional label for self-supervised vision transformer," in *Proc. 37th AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 3, pp. 3516–3524, doi: [10.1609/aaai.v37i3.25461](https://doi.org/10.1609/aaai.v37i3.25461).
- [29] Z. Song, Y. Xu, Z. He, L. Jiang, N. Jing, and X. Liang, "Cp-vit: Cascade vision transformer pruning via progressive sparsity prediction," 2022, *arXiv:2203.04570*.
- [30] E. Cengil and A. Çınar, "A new approach for image classification: Convolutional neural network," *Eur. J. Techn.*, vol. 6, no. 2, pp. 96–103, 2016.
- [31] Y. Wang, R. Huang, S. Song, Z. Huang, and G. Huang, "Not all images are worth 16×16 words: Dynamic transformers for efficient image recognition," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 11960–11973.
- [32] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [33] C. Meng et al., "CTA-Net: A CNN-transformer aggregation network for improving multi-scale feature extraction," 2024, [Online]. Available: <https://api.semanticscholar.org/CorpusID:273351365>
- [34] Z. Li et al., "SAViT: Structure-aware vision transformer pruning via collaborative optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, New Orleans, Louisiana, USA, vol. 35, 2022, pp. 9010–9023.
- [35] Z. Mo, H. Shi, and S. J. Pan, "Probabilistic neural pruning via sparsity evolutionary fokker-planck-kolmogorov equation," in *Proc. Int. Conf. Learn. Representations*, May 2025, vol. 2025, pp. 71586–71618.
- [36] Z. Fu, "Vision transformer: Vit and its derivatives," 2022, *arXiv:2205.11239*.
- [37] H. Yu and J. Wu, "A unified pruning framework for vision transformers," *Sci. China Inf. Sci.*, vol. 66, no. 7, Jul. 2023, Art. no. 179101, doi: [10.1007/s11432-022-3646-6](https://doi.org/10.1007/s11432-022-3646-6).
- [38] B. Ahn and T. Kim, "Deeper weight pruning without accuracy loss in deep neural networks: Signed-digit representation-based approach," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 41, no. 3, pp. 656–668, Mar. 2022, doi: [10.1109/TCAD.2021.3064914](https://doi.org/10.1109/TCAD.2021.3064914).
- [39] X. Chen, "Escoinc: Efficient sparse convolutional neural network inference on gpus," 2018, *arXiv:1802.10280*.
- [40] D. Kuznedelev, E. Kurtic, E. Frantar, and D. Alistarh, "oViT: An accurate second-order pruning framework for vision transformers," 2022, *arXiv:2210.09223*.
- [41] Y. Le Cun, J. S. Denker, and S. A. Solla, "Optimal brain damage, advances in neural information processing systems," in *Denver 1989*, vol. 598, D. Touretzky, Ed. San Mateo, CA, USA: Morgan Kaufmann, 1990, pp. 598–605.
- [42] Y. Prasetyo, N. Yudistira, and A. W. Widodo, "Sparse then prune: Toward efficient vision transformers," 2023, *arXiv:2307.11988*.
- [43] S. Ahsaei and M. Raji, "LOST-ViT: A low overhead soft error tolerance framework for vision transformers via model compression and selective bit-level redundancy," *J. Syst. Archit.*, vol. 170, 2025, Art. no. 103623.
- [44] D. Shi, C. Tao, Y. Jin, Z. Yang, C. Yuan, and J. Wang, "UPop: Unified and progressive pruning for compressing vision-language transformers," in *Proc. Mach. Learn. Res.*, 2023, pp. 31292–31311.
- [45] Y. He and J. T. Zhou, "Data-independent module-aware pruning for hierarchical vision transformers," 2024, *arXiv:2404.13648*.
- [46] F. Moya Rueda, R. Grzeszick, and G. A. Fink, "Neuron pruning for compressing deep networks using maxout architectures," *Proc. German Conf. Pattern Recognit.*, 2017, pp. 177–188, doi: [10.1007/978-3-319-66709-6_15](https://doi.org/10.1007/978-3-319-66709-6_15).
- [47] K. Shim, J. Yun, and S. Choi, "SNP: Structured neuron-level pruning to preserve attention scores," in *Proc. Comput. Vis.—ECCV*, 2025, pp. 90–104.
- [48] S. Bai, J. Chen, Y. Yang, and Y. Liu, "Multi-dimension compression of feed-forward network in vision transformers," *Pattern Recognit. Lett.*, vol. 176, pp. 56–61, 2023, doi: [10.1016/j.patrec.2023.10.014](https://doi.org/10.1016/j.patrec.2023.10.014).
- [49] M. Zhu, Y. Tang, and K. Han, "Visual transformer pruning," in *Proc. KDD Workshop*, Apr. 2021, pp. 1–4.
- [50] A. Kumar, "Vision transformer compression with structured pruning and low rank approximation," 2022, *arXiv:2203.13444*.
- [51] E. Lee and Y. Hwang, "Automatic channel pruning for multi-head attention," 2024, *arXiv:2405.20867*.
- [52] K. Kinfu and R. Vidal, "Efficient vision transformer for human pose estimation via patch selection," in *Proc. Brit. Mach. Vis. Conf.*, 2023.
- [53] Y. Liang, C. Ge, Z. Tong, Y. Song, J. Wang, and P. Xie, "Not all patches are what you need: Expediting vision transformers via token reorganizations," in *Proc. 10th Int. Conf. Learn. Representations*, 2022, pp. 1–21.
- [54] Y. Rao, W. Zhao, B. Liu, J. Lu, J. Zhou, and C. J. Hsieh, "DynamicViT: Efficient vision transformers with dynamic token sparsification," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 13937–13949, 2021.
- [55] P. Dong et al., "HeatViT: Hardware-efficient adaptive token pruning for vision transformers," in *Proc. Int. Symp. High-Perform. Comput. Archit.*, 2023, pp. 442–455, doi: [10.1109/HPCA56546.2023.10071047](https://doi.org/10.1109/HPCA56546.2023.10071047).
- [56] Y. Tang et al., "Patch slimming for efficient vision transformers," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 12165–12174, doi: [10.1109/CVPR52688.2022.01185](https://doi.org/10.1109/CVPR52688.2022.01185).
- [57] Q. Tang, B. Zhang, J. Liu, F. Liu, and Y. Liu, "Dynamic token pruning in plain vision transformers for semantic segmentation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 777–786, doi: [10.1109/ICCV51070.2023.00078](https://doi.org/10.1109/ICCV51070.2023.00078).
- [58] X. Wu, F. Zeng, X. Wang, and X. Chen, "Ppt: Token pruning and pooling for efficient vision transformers," 2023, *arXiv:2310.01812*.
- [59] S. Wei, T. Ye, S. Zhang, Y. Tang, and J. Liang, "Joint token pruning and squeezing towards more aggressive compression of vision transformers," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 2092–2101, doi: [10.1109/CVPR52729.2023.00208](https://doi.org/10.1109/CVPR52729.2023.00208).
- [60] Y. Wang, B. Du, W. Wang, and C. Xu, "Multi-tailed vision transformer for efficient inference," *Neural Netw.*, vol. 174, Jun. 2024, Art. no. 106235, doi: [10.1016/j.neunet.2024.106235](https://doi.org/10.1016/j.neunet.2024.106235).
- [61] Y. Liu, M. Gehrig, N. Messikommer, M. Cannici, and D. Scaramuzza, "Revisiting token pruning for object detection and instance segmentation," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2024, pp. 2658–2668.
- [62] Y. Zhang, L. Wei, and N. Freris, "Synergistic patch pruning for vision transformer: Unifying intra- & inter-layer patch importance," in *Proc. 12th Int. Conf. Learn. Representations*, 2024, pp. 1–19.

- [63] X. Xu, S. Wang, Y. Chen, Y. Zheng, Z. Wei, and J. Liu, "GTP-ViT: Efficient vision transformers via graph-based token propagation," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2024, pp. 86–95.
- [64] L. E. E. Jongho and H. Kim, "DCT-ViT: High-frequency pruned vision transformer with discrete cosine transform," *IEEE Access*, vol. 12, pp. 80386–80396, 2024.
- [65] M. Kim, S. Gao, Y.-C. Hsu, Y. Shen, and H. Jin, "Token fusion: Bridging the gap between token pruning and token merging," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2024, pp. 1372–1381.
- [66] Z. Kong et al., "SPViT: Enabling faster vision transformers via latency-aware soft token pruning," *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 620–640, doi: [10.1007/978-3-031-20083-0_37](https://doi.org/10.1007/978-3-031-20083-0_37).
- [67] H. Ye et al., "Once for both: Single stage of importance and sparsity search for vision transformer compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 5578–5588.
- [68] A. Wang, H. Chen, Z. Lin, S. Zhao, J. Han, and G. Ding, "Cait: Triple-win compression towards high accuracy, fast inference, and favorable transferability for vits," 2023, *arXiv:2309.15755*.
- [69] D. Parikh, S. Li, B. Zhang, R. Kannan, C. Busart, and V. Prasanna, "Accelerating ViT inference on FPGA through static and dynamic pruning," in *Proc. IEEE 32nd Annu. Int. Sympos. Field-Programm. Custom Comput. Mach.*, Orlando, FL, USA, vol. 34, 2024, pp. 78–89.
- [70] Y. Li et al., "How does attention work in vision transformers? A visual analytics attempt," *IEEE Trans. Vis. Comput. Graph.*, vol. 29, no. 6, pp. 2888–2900, Jun. 2023, doi: [10.1109/TVCG.2023.3261935](https://doi.org/10.1109/TVCG.2023.3261935).
- [71] L. Yu and W. Xiang, "X-Pruner: Explainable pruning for vision transformers," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 24355–24363, doi: [10.1109/CVPR52729.2023.02333](https://doi.org/10.1109/CVPR52729.2023.02333).
- [72] M. Yin, B. Uzkent, Y. Shen, H. Jin, and B. Yuan, "GOHSP: A unified framework of graph and optimization-based heterogeneous structured pruning for vision transformer," in *Proc. Conf. AAAI Artif. Intell.*, vol. 37, no. 9, Jun. 2023, pp. 10954–10962.
- [73] S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, "Transformers in vision: A survey," *ACM Comput. Surv.*, vol. 54, no. 10, 2022, Art. 200, doi: [10.1145/3505244](https://doi.org/10.1145/3505244).
- [74] F. Yu, K. Huang, M. Wang, Y. Cheng, W. Chu, and L. Cui, "Width & depth pruning for vision transformers," *AAAI*, vol. 36, no. 3, pp. 3143–3151, Jun. 2022.
- [75] H. Yang, H. Yin, M. Shen, P. Molchanov, H. Li, and J. Kautz, "Global vision transformer pruning with Hessian-aware saliency," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 18547–18557, doi: [10.1109/CVPR52729.2023.01779](https://doi.org/10.1109/CVPR52729.2023.01779).
- [76] P. Wimmer, J. Mehnert, and A. P. Condurache, "Dimensionality reduced training by pruning and freezing parts of a deep neural network: A survey," *Artif. Intell. Rev.*, vol. 56, no. 12, pp. 14257–14295, 2023, doi: [10.1007/s10462-023-10489-1](https://doi.org/10.1007/s10462-023-10489-1).
- [77] A. Balasubramaniam, F. Sunny, and S. Pasricha, "R-TOSS: A framework for real-time object detection using semi-structured pruning," in *Proc.-Des. Automat. Conf.*, 2023, pp. 1–6, doi: [10.1109/DAC56929.2023.10247917](https://doi.org/10.1109/DAC56929.2023.10247917).
- [78] Z. Long, R. McCreadie, G. A. Camarasa, and Z. Meng, "LaCViT: A label-aware contrastive fine-tuning framework for vision transformers," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2024, pp. 5275–5279.
- [79] R. Kumar, M. K. Sarker, and S. R. Islam, "Vision transformers for galaxy morphology classification: Fine-tuning pre-trained networks vs. training from scratch," in *Proc. Int. Conf. Deep Learn. Theory Appl.*, 2023, pp. 115–125, doi: [10.1007/978-3-031-39059-3_8](https://doi.org/10.1007/978-3-031-39059-3_8).
- [80] M. Rajpal, Y. Zhang, and B. K. H. Low, "Pruning during training by network efficacy modeling," *Mach. Learn.*, vol. 112, no. 7, pp. 2653–2684, 2023, doi: [10.1007/s10994-023-06304-1](https://doi.org/10.1007/s10994-023-06304-1).
- [81] S. Lym, E. Choukse, S. Zangeneh, W. Wen, S. Sanghavi, and M. Erez, "PruneTrain: Fast neural network training by dynamic sparse model reconfiguration," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2019, doi: [10.1145/3295500.3356156](https://doi.org/10.1145/3295500.3356156).
- [82] S. Roy, P. Panda, G. Srinivasan, and A. Raghunathan, "Pruning filters while training for efficiently optimizing deep learning networks," in *Proc. Int. Joint Conf. Neural Netw.*, 2020, pp. 1–7, doi: [10.1109/IJCNN48605.2020.9207588](https://doi.org/10.1109/IJCNN48605.2020.9207588).
- [83] H. Guo, S. Li, B. Li, Y. Ma, and X. Ren, "A new learning automata-based pruning method to train deep neural networks," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3263–3269, Oct. 2018, doi: [10.1109/JIOT.2017.2711426](https://doi.org/10.1109/JIOT.2017.2711426).
- [84] M. V. dos Santos Neto, N. F. F. da Silva, and A. da Silva Soares, "A survey and study impact of tweet sentiment analysis via transfer learning in low resource scenarios," *Lang. Resour. Eval.*, vol. 58, no. 1, pp. 133–174, 2024, doi: [10.1007/s10579-023-09687-8](https://doi.org/10.1007/s10579-023-09687-8).
- [85] Z. Lu, H. Xie, C. Liu, and Y. Zhang, "Bridging the gap between vision transformers and convolutional neural networks on small datasets," *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 14663–14677, 2022.
- [86] H. Ide and T. Kurita, "Improvement of learning for CNN with ReLU activation by sparse regularization," in *Proc. Int. Joint Conf. Neural Netw.*, 2017, pp. 2684–2691, doi: [10.1109/IJCNN.2017.7966185](https://doi.org/10.1109/IJCNN.2017.7966185).
- [87] E. Tartaglione, A. Fiandrotti, S. Lepsøy, and G. Francini, "Learning sparse neural networks via sensitivity-driven regularization," *Adv. Neural Inf. Process. Syst.*, 2018, pp. 1–11.
- [88] J. Liu, Z. Xu, R. Shi, R. C. C. Cheung, and H. K. H. So, "Dynamic sparse training: Find efficient sparse network from scratch with trainable masked layers," in *Proc. 8th Int. Conf. Learn. Representations*, 2020, pp. 1–14.
- [89] G. Jo, G. Lee, and D. Shin, "Exploring group sparsity using dynamic sparse training," in *Proc. IEEE Int. Conf. Consum. Electron.*, 2020, pp. 1–2, doi: [10.1109/ICCE-Asia49877.2020.9277137](https://doi.org/10.1109/ICCE-Asia49877.2020.9277137).
- [90] A. Nowak, B. Grooten, D. C. Mocanu, and J. Tabor, "Fantastic weights and how to find them: Where to prune in dynamic sparse training," *Adv. Neural Inf. Process. Syst.*, vol. 36, pp. 55160–55192, 2024.
- [91] S. A. Aketi, S. Roy, A. Raghunathan, and K. Roy, "Gradual channel pruning while training using feature relevance scores for convolutional neural networks," *IEEE Access*, vol. 8, pp. 171924–171932, 2020, doi: [10.1109/ACCESS.2020.3024992](https://doi.org/10.1109/ACCESS.2020.3024992).
- [92] C. Yu, T. Chen, Z. Gan, and J. Fan, "Boost vision transformer with GPU-friendly sparsity and quantization," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 22658–22668, doi: [10.1109/CVPR52729.2023.02170](https://doi.org/10.1109/CVPR52729.2023.02170).
- [93] Y. J. Zheng, S. B. Chen, C. H. Q. Ding, and B. Luo, "Model compression based on differentiable network channel pruning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 12, pp. 10203–10212, Dec. 2023, doi: [10.1109/TNNLS.2022.3165123](https://doi.org/10.1109/TNNLS.2022.3165123).
- [94] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. 4th Int. Conf. Learn. Representations*, 2016, pp. 1–14.
- [95] K. An and Y. Zhang, "LPViT: A transformer based model for PCB image classification and defect detection," *IEEE Access*, vol. 10, pp. 42542–42553, 2022, doi: [10.1109/ACCESS.2022.3168861](https://doi.org/10.1109/ACCESS.2022.3168861).
- [96] Y. Rao, J. Lu, J. Lin, and J. Zhou, "Runtime network routing for efficient image classification," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 10, pp. 2291–2304, Oct. 2019, doi: [10.1109/TPAMI.2018.2878258](https://doi.org/10.1109/TPAMI.2018.2878258).
- [97] R. J. N. Baldock, H. Maennel, and B. Neyshabur, "Deep learning through the lens of example difficulty," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 10876–10889, 2021.
- [98] R. T. Mullapudi, W. R. Mark, N. Shazeer, and K. Fatahalian, "HydraNets: Specialized dynamic architectures for efficient inference," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8080–8089, doi: [10.1109/CVPR.2018.00843](https://doi.org/10.1109/CVPR.2018.00843).
- [99] S. Elkerdawy, M. Elhoushi, H. Zhang, and N. Ray, "Fire together wire together: A dynamic pruning approach with self-supervised mask prediction," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 12444–12453, doi: [10.1109/CVPR52688.2022.01213](https://doi.org/10.1109/CVPR52688.2022.01213).
- [100] R. Shi, T. Li, L. Zhang, and Y. Yamaguchi, "Visualization comparison of vision transformers and convolutional neural networks," *IEEE Trans. Multimedia*, vol. 26, pp. 2327–2339, 2024, doi: [10.1109/TMM.2023.3294805](https://doi.org/10.1109/TMM.2023.3294805).
- [101] K. Shibata, T. Ejima, Y. Tokumaru, and T. Matsuki, "Sensitivity – Local index to control chaoticity or gradient globally," *Neural Netw.*, vol. 143, pp. 436–451, 2021, doi: [10.1016/j.neunet.2021.06.015](https://doi.org/10.1016/j.neunet.2021.06.015).
- [102] D. Shan, X. Zhang, and C. Zhang, "Visual saliency based on extended manifold ranking and third-order optimization refinement," *Pattern Recognit. Lett.*, vol. 116, pp. 1–7, 2018, doi: [10.1016/j.patrec.2018.09.002](https://doi.org/10.1016/j.patrec.2018.09.002).
- [103] C. G. Petra, F. Qiang, M. Lubin, and J. Huchette, "On efficient Hessian computation using the edge pushing algorithm in Julia," *Optim. Methods Softw.*, vol. 33, no. 4–6, pp. 1010–1029, 2018, doi: [10.1080/10556788.2018.1480625](https://doi.org/10.1080/10556788.2018.1480625).

- [104] F. Cao, S. C. Han, S. Long, C. Xu, and J. Poon, "Understanding attention for vision-and-language tasks," in *Proc.-Int. Conf. Comput. Linguistics*, 2022, pp. 3438–3453.
- [105] H. J. Escalante, "Automated machine learning—A brief review at the end of the early years," in *Natural Computing Series*. Cham: Springer, 2021, pp. 11–28.
- [106] C. White et al., "Neural architecture search: Insights from 1000 papers," 2023, *arXiv:2301.08727*.
- [107] A. Awasthi and S. Sarawagi, "Continual learning with neural networks: A review," in *Proc. ACM India Joint Int. Conf. Data Sci. Manage. Data*, 2019, pp. 362–365, doi: [10.1145/3297001.3297062](https://doi.org/10.1145/3297001.3297062).
- [108] Z. Chen and B. Liu, "Lifelong machine learning," *Synth. Lect. Artif. Intell. Mach. Learn.*, vol. 10, no. 3, pp. 1–145, Nov. 2016.
- [109] M. Sultan, "Overview of federated learning," *Int. Res. J. Moderniz. Eng. Technol. Sci.*, vol. 5, no. 3, pp. 1–14, 2024.
- [110] C.-H. Ho and N. Vasconcelos, "Contrastive learning with adversarial examples," in *Proc. Adv. Neural Inf. Process. Syst.*, Oct. 2020, pp. 17081–17081.
- [111] L. Xu, H. Song, L. Tian, Z. Wang, and M. Wang, "TAFP-ViT: A transformer accelerator via QKV computational fusion and adaptive pruning for vision transformer," *ACM Trans. Embedded Comput. Syst.*, vol. 24, no. 5, 2025, Art. no. 78, doi: [10.1145/3745028](https://doi.org/10.1145/3745028).
- [112] M. Marchetti, D. Traini, D. Ursino, and L. Virgili, "Efficient token pruning in vision transformers using an attention-based multilayer network," *Expert Syst. Appl.*, vol. 279, 2025, Art. no. 127449.
- [113] J. Chen, S. Deng, W. Yu, and L. Wei, "RAAG: Redundancy-adaptive and attention-guided token pruning for efficient video action detection," *Neurocomputing*, vol. 658, 2025, Art. no. 131615, doi: [10.1016/j.neucom.2025.131615](https://doi.org/10.1016/j.neucom.2025.131615).
- [114] S. Sarkar, G. Datta, S. Kundu, K. Zheng, C. Bhattacharyya, and P. A. Beerel, "MaskVD: Region masking for efficient video object detection," in *Proc.-IEEE Winter Conf. Appl. Comput. Vis.*, 2025, pp. 1955–1964, doi: [10.1109/WACV61041.2025.00197](https://doi.org/10.1109/WACV61041.2025.00197).